

DRAGON SLAYING GUIDE

Bug Hunting In VMware Device Virtualization



Speakers: JiaQing Huang, Hao Zheng, Yue Liu
From TianGong Team of Legendsec at QI-ANXIN Group

Who We Are



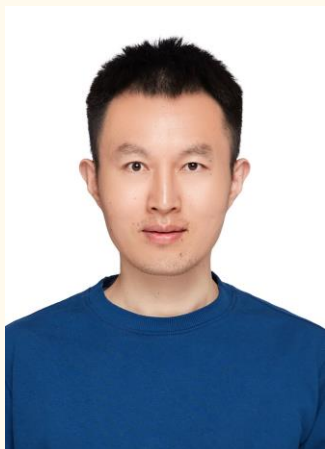
JiaQing Huang

- **Researcher of TianGong Team**
- **Email: njfu.s0duku@gmail.com**
- **Twitter: @S0dukuN**



Hao Zheng

- **Researcher of TianGong Team**
- **Email: ted142890@gmail.com**
- **Twitter: @zhz_xihelab**



Yue Liu

- **Team leader of TianGong Team**
- **Email: liuyue0310@gmail.com**
- **Twitter: @Mr_LiuYue**



Twitter: TianGongLab

WeChat Account: 奇安信天工实验室

<https://tiangonglab.github.io/blog/>



The Story

When ?

- **Started to research virtualization security at the end of 2022**
- **Basically, we know nothing about virtualization, totally rookies**

Why ?

- **Virtualization is involved in a lot of hacking contests**
- **It's hard, it's complex, and the best motivation for me to do this is just
I WANT TO UNDERSTAND IT**

Achievements

- **We successfully escaped from the Parallels Desktop at GeekCon 2023**
- **We reported a lot of VMware Workstation\ESXi bugs**
- **We learned a lot -- not just about virtualization, but about the whole architecture of the entire computer (Kernel, Memory, Devices ...)**



The Topics We're Discussing

- **VMware Hypervisor Reverse Engineering**

VMware virtualization architecture

- **VMware Device Virtualization Bug Hunting**

USB Virtualization Bug Hunting

SCSI Virtualization Bug Hunting





VMware Hypervisor Reverse Engineering



DEEROTDUN

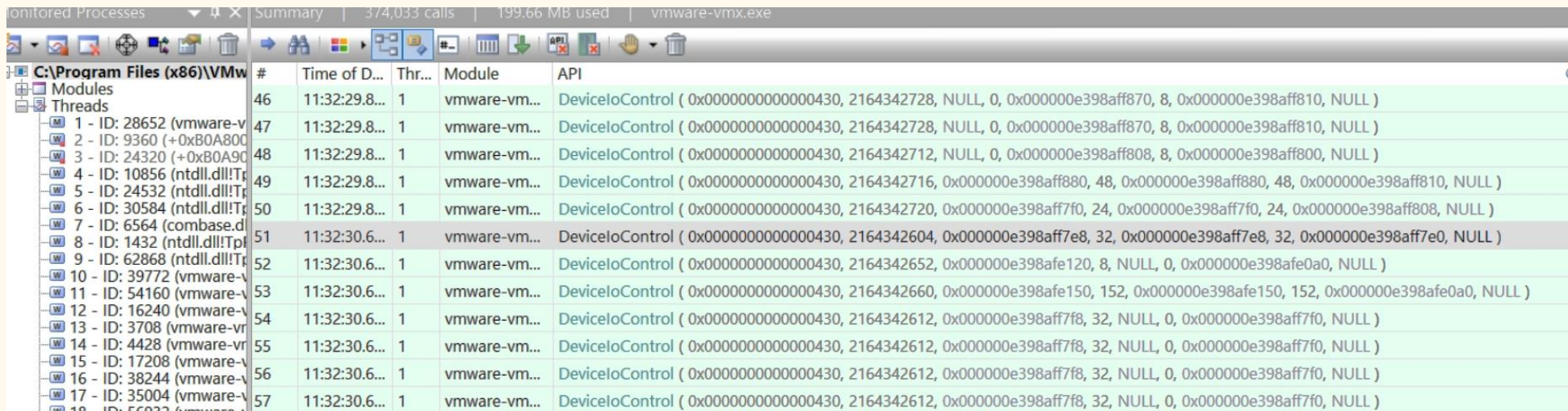
Tips For Beginners

- **VMware Workstation's main executable vmware-vmx.exe is basically identical to vmx on ESXi — there are functions that wrap the System APIs**
- **Debugging VMware Workstation under Windows is even easier. So, for this part, we will use Workstation (vmware-vmx.exe) as an example**
- **Symbols from open-vm-tools can be very helpful, because vmx is based on event loop. Open-vm-tools includes these functions implementation**
- **Virtualization/OS/Hardware technique papers, blogs, CVEs. You can even get ideas from Qemu**



VMX Main Loop

- **vmx should have a loop that handles the switch in/out of virtual machine**
- **Generally, it should access kernel for switch in, so there should be a kernel module**
- **We can utilize API Monitor to monitor the communication process between vmx and the kernel module vmx86.sys**



monitored Processes | Summary | 374,033 calls | 199.66 MB used | vmware-vmx.exe

	#	Time of D...	Thr...	Module	API
1 - ID: 28652 (vmware-v...	46	11:32:29.8...	1	vmware-vm...	DeviceIoControl (0x0000000000000430, 2164342728, NULL, 0, 0x0000000e398aff870, 8, 0x0000000e398aff810, NULL)
2 - ID: 9360 (+0xB0A800...	47	11:32:29.8...	1	vmware-vm...	DeviceIoControl (0x0000000000000430, 2164342728, NULL, 0, 0x0000000e398aff870, 8, 0x0000000e398aff810, NULL)
3 - ID: 24320 (+0xB0A90...	48	11:32:29.8...	1	vmware-vm...	DeviceIoControl (0x0000000000000430, 2164342712, NULL, 0, 0x0000000e398aff808, 8, 0x0000000e398aff800, NULL)
4 - ID: 10856 (ntdll.dll!T...	49	11:32:29.8...	1	vmware-vm...	DeviceIoControl (0x0000000000000430, 2164342716, 0x0000000e398aff880, 48, 0x0000000e398aff880, 48, 0x0000000e398aff810, NULL)
5 - ID: 24532 (ntdll.dll!T...	50	11:32:29.8...	1	vmware-vm...	DeviceIoControl (0x0000000000000430, 2164342720, 0x0000000e398aff7f0, 24, 0x0000000e398aff7f0, 24, 0x0000000e398aff808, NULL)
6 - ID: 30584 (ntdll.dll!T...	51	11:32:30.6...	1	vmware-vm...	DeviceIoControl (0x0000000000000430, 2164342604, 0x0000000e398aff7e8, 32, 0x0000000e398aff7e8, 32, 0x0000000e398aff7e0, NULL)
7 - ID: 6564 (combase.d...	52	11:32:30.6...	1	vmware-vm...	DeviceIoControl (0x0000000000000430, 2164342652, 0x0000000e398afe120, 8, NULL, 0, 0x0000000e398afe0a0, NULL)
8 - ID: 1432 (ntdll.dll!T...	53	11:32:30.6...	1	vmware-vm...	DeviceIoControl (0x0000000000000430, 2164342660, 0x0000000e398afe150, 152, 0x0000000e398afe150, 152, 0x0000000e398afe0a0, NULL)
9 - ID: 62868 (ntdll.dll!T...	54	11:32:30.6...	1	vmware-vm...	DeviceIoControl (0x0000000000000430, 2164342612, 0x0000000e398aff7f8, 32, NULL, 0, 0x0000000e398aff7f0, NULL)
10 - ID: 39772 (vmware-v...	55	11:32:30.6...	1	vmware-vm...	DeviceIoControl (0x0000000000000430, 2164342612, 0x0000000e398aff7f8, 32, NULL, 0, 0x0000000e398aff7f0, NULL)
11 - ID: 54160 (vmware-vr...	56	11:32:30.6...	1	vmware-vm...	DeviceIoControl (0x0000000000000430, 2164342612, 0x0000000e398aff7f8, 32, NULL, 0, 0x0000000e398aff7f0, NULL)
12 - ID: 16240 (vmware-v...	57	11:32:30.6...	1	vmware-vm...	DeviceIoControl (0x0000000000000430, 2164342612, 0x0000000e398aff7f8, 32, NULL, 0, 0x0000000e398aff7f0, NULL)
13 - ID: 3708 (vmware-vr...					
14 - ID: 4428 (vmware-vr...					
15 - ID: 17208 (vmware-v...					
16 - ID: 38244 (vmware-v...					
17 - ID: 35004 (vmware-v...					
18 - ID: 55932 (vmware-v...					



VMX Main Loop

- **There are log strings in vmx, which can be helpful for you to understand IOCTL code meaning**

```
__int64 __fastcall IOCTL_VMX86_RUN_VM(int a1)
{
    const char *v1; // rax
    DWORD BytesReturned; // [rsp+40h] [rbp-18h] BYREF

    if ( DeviceIoControl(hDevice, 0x81013F67, (LPVOID)(unsigned int)(a1 + 1000), 8u, 0i64, 0, &BytesReturned, 0i64) )
        return BytesReturned;
    v1 = (const char *)W32Util_LastError2Msg();
    Warning_("IOCTL_VMX86_RUN_VM failed: %s\n", v1);
    return 0xFFFFFFFFi64;
}
```



VMX Main Loop

- When you locate the loop code, you should see the UserRpc handlers table which is the UserRpc implementation on vmx's side

```
userRpcBlock = SharedArea_Lookup(a1, "userRpcBlock", 0x1088i64);
v3 = (__int64 *)SharedArea_Lookup(v1, "monitorSwitchError", 8i64);
do
{
    Id = IOCTL_VMX86_RUN_VM(v1);          // 1. ioctl vmx86.sys to switch to vmm
                                          // 2. receive the UserRpcHandler Id
    if ( (*(_DWORD *)qword_7FF7CF869570 >= 2u )
        MonitorLogMonitorPanic();
    if ( ((Id + 0x80000000) & 0x80000000) == 0 && Id != -8193 )
    {
        _mm_lfence();
        Panic("VCPUs RunVM failed: %d.\n", v1, Id);
    }
    v5 = *v3;
    v6 = *v3;
    if ( Id == -8193 && v5 != -1 )
    {
        v6 = sub_7FF7CE3B8120(v1);
        *v3 = v6;
    }
    if ( v6 && v5 != -1 )
    {
        v8 = sub_7FF7CE978910(v6);
        LOBYTE(v9) = 1;
        v10 = (const char *)v8;
        sub_7FF7CE966A00(v9);
        Panic("%s\n", v10);
    }
    if ( (*(_DWORD *)qword_7FF7CF869570 >= 2u )
        MonitorLogMonitorPanic();
    result = Monitor_ProcessUserRpcCall((__int64)VMContext, userRpcBlock, Id); // 1. call UserRpcHandler by Id
                                          // 2. call UserRpcHandler with userRpcBlock shared area.
}
while ( Id != 305 );
```

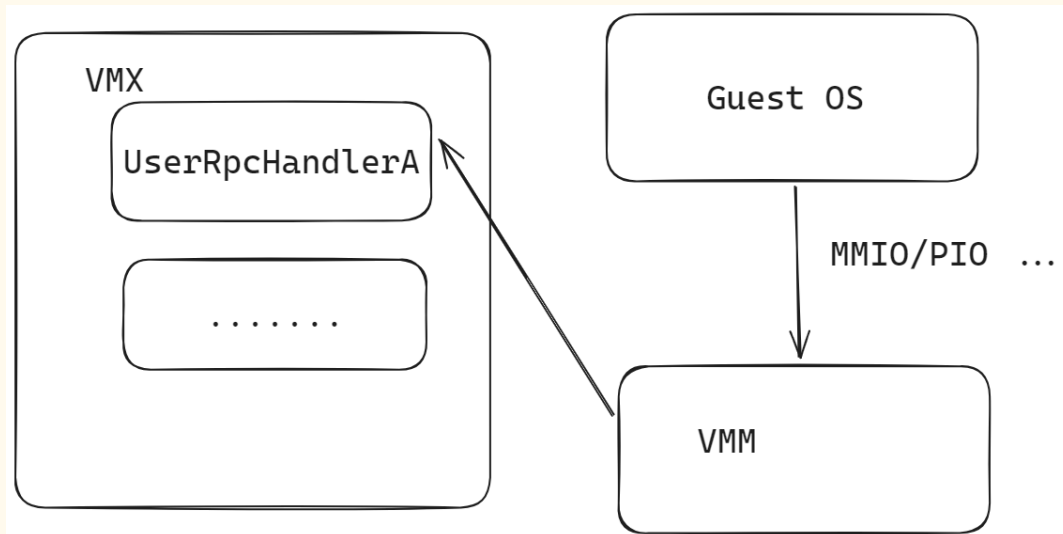
```
.data:00007FF7CF016980
.data:00007FF7CF016990
.data:00007FF7CF0169A0
.data:00007FF7CF0169B0
.data:00007FF7CF0169C0
.data:00007FF7CF0169D0
.data:00007FF7CF0169E0
.data:00007FF7CF0169F0
.data:00007FF7CF016A00
.data:00007FF7CF016A10
.data:00007FF7CF016A20
.data:00007FF7CF016A30
.data:00007FF7CF016A40
.data:00007FF7CF016A50
.data:00007FF7CF016A60
.data:00007FF7CF016A70
.data:00007FF7CF016A80
.data:00007FF7CF016A90
.data:00007FF7CF016AA0
.data:00007FF7CF016AB0
.data:00007FF7CF016AC0
.data:00007FF7CF016AD0
.data:00007FF7CF016AE0
.data:00007FF7CF016AF0
.data:00007FF7CF016B00
.data:00007FF7CF016B10
.data:00007FF7CF016B20
.data:00007FF7CF016B30
.data:00007FF7CF016B40
.data:00007FF7CF016B50
.data:00007FF7CF016B60
.data:00007FF7CF016B70
.data:00007FF7CF016B80
.data:00007FF7CF016B90
.data:00007FF7CF016BA0
.data:00007FF7CF016BB0
```

```
; DATA XREF: Monitor_ProcessUserRpcCall+
UserRpcCallHandler <offset _guard_check_icall_nop, 0>; 1; Micro
UserRpcCallHandler <offset sub_7FF7CE981E90, 0>; 2
UserRpcCallHandler <offset sub_7FF7CE981E70, 0>; 3
UserRpcCallHandler <offset sub_7FF7CE981E80, 0>; 4
UserRpcCallHandler <offset sub_7FF7CE981BC0, 0>; 5
UserRpcCallHandler <offset sub_7FF7CE981EA0, 0>; 6
UserRpcCallHandler <offset sub_7FF7CE981EC0, 1>; 7
UserRpcCallHandler <offset sub_7FF7CE981AF0, 1>; 8
UserRpcCallHandler <offset sub_7FF7CE981B60, 1>; 9
UserRpcCallHandler <offset sub_7FF7CE981E00, 1>; 10
UserRpcCallHandler <offset j_MonitorLogMonitorPanic, 0>; 11
UserRpcCallHandler <offset sub_7FF7CE978CF0, 0>; 12
UserRpcCallHandler <offset sub_7FF7CE966A00, 0>; 13
UserRpcCallHandler <offset sub_7FF7CE966B00, 0>; 14
UserRpcCallHandler <offset MonitorLoop_FinalizeHandler, 1>; 15
UserRpcCallHandler <offset sub_7FF7CE96D0C0, 1>; 16
UserRpcCallHandler <offset sub_7FF7CE965190, 0>; 17
UserRpcCallHandler <offset sub_7FF7CE965900, 1>; 18
UserRpcCallHandler <offset sub_7FF7CE980FD0, 0>; 19
UserRpcCallHandler <offset unknown_libname_17, 0>; 20
UserRpcCallHandler <offset sub_7FF7CE462770, 0>; 21
UserRpcCallHandler <offset sub_7FF7CE45CDD0, 0>; 22
UserRpcCallHandler <offset sub_7FF7CE976F80, 1>; 23
UserRpcCallHandler <offset sub_7FF7CE4DAE00, 1>; 24
UserRpcCallHandler <offset sub_7FF7CE981F70, 0>; 25
UserRpcCallHandler <offset sub_7FF7CE96BFF0, 1>; 26
UserRpcCallHandler <offset sub_7FF7CE491C00, 0>; 27
UserRpcCallHandler <offset sub_7FF7CE3BB8D0, 1>; 28
UserRpcCallHandler <offset sub_7FF7CE4840C0, 1>; 29
UserRpcCallHandler <offset sub_7FF7CE4430F0, 1>; 30
UserRpcCallHandler <offset sub_7FF7CE4C0340, 1>; 31
UserRpcCallHandler <offset Vmxnet3_RPCHandler, 1>; 32
UserRpcCallHandler <offset sub_7FF7CE4F3BD0, 1>; 33
UserRpcCallHandler <offset Xhci_RPCHandler, 1>; 34
UserRpcCallHandler <offset PVSCSI_RPCHandler, 0>; 35
```



UserRPC In VMX

- **VMware designed UserRPC for vmm to interact with the vmx on host**
- **Similar to Hypercall, but on userspace vmx**
- **UserRPC contains a lot of code related to device emulation**
- **A lot of bugs that are found in device emulation functions are called from related UserRpcHandler**



UserRPC In VMX

- **UserRPCHandler** only took one param: **UserRpcBlock** pointer
- **UserRpcBlock** is a **SharedArea** memory region

```
userRpcBlock = SharedArea_Lookup(a1, "userRpcBlock", 0x1088i64);  
v3 = (__int64 *)SharedArea_Lookup(v1, "monitorSwitchError", 8i64);  
do  
r
```

```
void __fastcall NVME_RPCHandler(__int64 rpcBlock)  
{  
    NvmeAdapterDeviceObject *adapterObject; // rsi  
    unsigned int v3; // r15d  
    NvmeShadredObj *NvmeShadredObj; // rbx  
    unsigned int BackendSCSIAdapterID; // edi  
    VStorageDeviceobject *v6; // rax  
    VStorageDeviceobject *v7; // rbx  
    unsigned int v8; // eax  
    char *v9; // rax
```

RPC Params



SharedArea Details

Reverse engineering tips:

- **The implementation of SharedArea is closely related to the creation and loading of vmm**
- **Trace interactions between vmware-vmx.exe and vmx86.sys to reverse engineer the virtual machine initialization**
- **Start from the SharedArea initialization process. Pay attention to memory allocation calls**



SharedArea Details

- **ELF Object - vmmblob is embedded in vmx**
- **ELF linker code within vmx**
- **vmm extensions stored in vmmblob's sections in format of ELF Object**
- **vmmblob and vmm extensions will be relinked to a new ELF for vmm in memory according to '.vmx' configuration**

```
root@debian:~# readelf -W -S vmmblob175-vmmmods.bin
There are 24 section headers, starting at offset 0x327990:

Section Headers:
[Nr] Name                Type              Address            Off    Size  ES Flg Lk Inf Al
[ 0]                      NULL              0000000000000000  000000 000000 00      0  0  0
[ 1] vmm.vmm                 PROGBITS          0000000000000000  000040 1c54d0 00      0  0  1
[ 2] vrdma-vrdma.vmm        PROGBITS          0000000000000000  1c5510 001a38 00      0  0  1
[ 3] vprobe-vprobe.vmm      PROGBITS          0000000000000000  1c6f48 021690 00      0  0  1
[ 4] vprobe-none.vmm        PROGBITS          0000000000000000  1e85d8 001170 00      0  0  1
[ 5] vmxnet3-vmxnet3.vmm    PROGBITS          0000000000000000  1e9748 002050 00      0  0  1
[ 6] vmce-vmce.vmm          PROGBITS          0000000000000000  1eb798 002b10 00      0  0  1
[ 7] vmce-none.vmm          PROGBITS          0000000000000000  1ee2a8 0013f8 00      0  0  1
[ 8] viommu-vvtd.vmm        PROGBITS          0000000000000000  1ef6a0 00c200 00      0  0  1
[ 9] viommu-none.vmm        PROGBITS          0000000000000000  1fb8a0 001360 00      0  0  1
[10] viommu-amd.vmm         PROGBITS          0000000000000000  1fcc00 009e98 00      0  0  1
[11] qat-qat.vmm            PROGBITS          0000000000000000  206a98 0024b0 00      0  0  1
[12] pvscsi-pvscsi.vmm      PROGBITS          0000000000000000  208f48 0031e0 00      0  0  1
[13] pcip-pcip.vmm          PROGBITS          0000000000000000  20c128 00a5c8 00      0  0  1
[14] nvme-nvme.vmm          PROGBITS          0000000000000000  2166f0 004778 00      0  0  1
[15] hv-vt.vmm              PROGBITS          0000000000000000  21ae68 078440 00      0  0  1
[16] hv-svm.vmm             PROGBITS          0000000000000000  2932a8 063b08 00      0  0  1
[17] gphys-npt.vmm          PROGBITS          0000000000000000  2f6db0 0121a8 00      0  0  1
[18] gphys-ept.vmm          PROGBITS          0000000000000000  308f58 012e10 00      0  0  1
[19] callstack-none.vmm     PROGBITS          0000000000000000  31bd68 000f88 00      0  0  1
[20] callstack-callstack.vmm PROGBITS          0000000000000000  31ccf0 000f88 00      0  0  1
[21] buslogic-buslogic.vmm  PROGBITS          0000000000000000  31dc78 001728 00      0  0  1
[22] ahci-ahci.vmm          PROGBITS          0000000000000000  31f3a0 008490 00      0  0  1
[23] .shstrtab              STRTAB            0000000000000000  327830 00015e 00      0  0  1
```



SharedArea Details

- **Predefined export symbols such as userRpcBlock and their corresponding mapping addresses in the .shared_per_vcpu_vmx section of the vmmblob**
- **Some virtual device implementations require shared memory, too. In the Elf Object sections related to SharedArea, vmx will define the export symbol in it**
- **vmx calculates the total size of SharedArea memories and allocates corresponding memory space**

```
.shared_per_vcpu_vmx:FFFFFFF8E88A388 monitorSwitchError dq ?  
.shared_per_vcpu_vmx:FFFFFFF8E88A390 align 20h  
.shared_per_vcpu_vmx:FFFFFFF8E88A3A0 public userRpcBlock  
.shared_per_vcpu_vmx:FFFFFFF8E88A3A0 userRpcBlock db ? ;  
.shared_per_vcpu_vmx:FFFFFFF8E88A3A0  
.shared_per_vcpu_vmx:FFFFFFF8E88A3A1 db ? ;  
.shared_per_vcpu_vmx:FFFFFFF8E88A3A2 db ? ;
```

```
if ( (unsigned __int8)SharedArea_PowerOnInt() )  
{  
    byte_141341128 = 0;  
    sharedAreaBase = Mem_Map(  
        49i64,  
        (unsigned int)(dword_1415B8CF4  
            + dword_1413410CC  
            + *((_DWORD *)qword_141384470 + 44)  
            * (dword_1413410F4 + dword_1413410A4 + dword_14134111C)),  
        0i64);  
    if ( sharedAreaBase )  
        goto LABEL_3;  
    sub_1406AF0A0(3i64, "@&!*!*(msg.sharedArea.noMappedMem)Failed to allocate shared memory.\n");  
}
```



SharedArea Details

- **.host_params** section of vmmblob contains vmm's GDT information
- **.monloader** section of vmmblob contains vmm's virtual address mapping information

```
.host_params:FFFFFFFFEC8D048      dw 0E9h          ; gdtInit.entries.index
.host_params:FFFFFFFFEC8D04A      db 2 dup(0)
.host_params:FFFFFFFFEC8D04C      dd 0            ; gdtInit.entries.base
.host_params:FFFFFFFFEC8D050      dd 0FFFFFFh     ; gdtInit.entries.limit
.host_params:FFFFFFFFEC8D054      dd 0Ah          ; gdtInit.entries.type
.host_params:FFFFFFFFEC8D058      dd 1            ; gdtInit.entries.S
.host_params:FFFFFFFFEC8D05C      dd 0            ; gdtInit.entries.DPL
.host_params:FFFFFFFFEC8D060      dd 1            ; gdtInit.entries.present
.host_params:FFFFFFFFEC8D064      dd 1            ; gdtInit.entries.longmode
.host_params:FFFFFFFFEC8D068      dd 0            ; gdtInit.entries.DB
.host_params:FFFFFFFFEC8D06C      dd 1            ; gdtInit.entries.gran
.host_params:FFFFFFFFEC8D070      dw 0EAh         ; gdtInit.entries.index
.host_params:FFFFFFFFEC8D072      db 2 dup(0)
.host_params:FFFFFFFFEC8D074      dd 0            ; gdtInit.entries.base
.host_params:FFFFFFFFEC8D078      dd 0FFFFFFh     ; gdtInit.entries.limit
.host_params:FFFFFFFFEC8D07C      dd 2            ; gdtInit.entries.type
.host_params:FFFFFFFFEC8D080      dd 1            ; gdtInit.entries.S
.host_params:FFFFFFFFEC8D084      dd 0            ; gdtInit.entries.DPL
.host_params:FFFFFFFFEC8D088      dd 1            ; gdtInit.entries.present
.host_params:FFFFFFFFEC8D08C      dd 0            ; gdtInit.entries.longmode
.host_params:FFFFFFFFEC8D090      dd 1            ; gdtInit.entries.DB
.host_params:FFFFFFFFEC8D094      dd 1            ; gdtInit.entries.gran
.host_params:FFFFFFFFEC8D094      _host_params  ends
```

```
.monLoaderHeader:FFFFFFFFDE2F000 ; const MonLoaderHeader monLoaderHeader
.monLoaderHeader:FFFFFFFFDE2F000 monLoaderHeader dq 8675309E98675309h ; magic
.monLoaderHeader:FFFFFFFFDE2F000 ; DATA XREF: _start+551r
.monLoaderHeader:FFFFFFFFDE2F000 ; _start+BD1r
.monLoaderHeader:FFFFFFFFDE2F008      dd 48h          ; entrySize
.monLoaderHeader:FFFFFFFFDE2F00C      dd 17h          ; count
.monLoaderHeader:FFFFFFFFDE2F010      dw 748h         ; codeSelector
.monLoaderHeader:FFFFFFFFDE2F012      dq 0FFFFFFFFDE0000h ; codeEntrypoint
.monLoaderHeader:FFFFFFFFDE2F01A      dw 750h         ; stackSelector
.monLoaderHeader:FFFFFFFFDE2F01C      dq 0FFFFFFFFC40800h ; stackEntrypoint
.monLoaderHeader:FFFFFFFFDE2F024      dq 0FFFFFFFFC000h ; monStartLPN
.monLoaderHeader:FFFFFFFFDE2F02C      dq 0FFFFFFFFFFFFh ; monEndLPN
.monLoaderHeader:FFFFFFFFDE2F034 ; const MonLoaderEntry stru_FFFFFFFFDE2F034
.monLoaderHeader:FFFFFFFFDE2F034 stru_FFFFFFFFDE2F034 dd ML_CONTENT_ADDRSPACE ; [0].content
.monLoaderHeader:FFFFFFFFDE2F034 ; DATA XREF: _start+A31r
.monLoaderHeader:FFFFFFFFDE2F038      dd ML_SOURCE_NONE ; [0].source
.monLoaderHeader:FFFFFFFFDE2F038      dq 0FFFFFFFFC000h ; [0].monVPN
.monLoaderHeader:FFFFFFFFDE2F03C      dq 4000h         ; [0].monPages
.monLoaderHeader:FFFFFFFFDE2F04C      dq 3             ; [0].flags
.monLoaderHeader:FFFFFFFFDE2F054      dd 0             ; [0].allocs
.monLoaderHeader:FFFFFFFFDE2F058      db 4 dup(0)      ; 0
.monLoaderHeader:FFFFFFFFDE2F05C      dq 0             ; [0].blobSrc.offset
.monLoaderHeader:FFFFFFFFDE2F064      dq 0             ; [0].blobSrc.size
.monLoaderHeader:FFFFFFFFDE2F06C      dq 0             ; [0].bspOnly
.monLoaderHeader:FFFFFFFFDE2F074      dq 0             ; [0].subIndex
.monLoaderHeader:FFFFFFFFDE2F07C      dd ML_CONTENT_PAGETABLE_L4 ; [1].content
.monLoaderHeader:FFFFFFFFDE2F080      dd ML_SOURCE_NONE ; [1].source
.monLoaderHeader:FFFFFFFFDE2F084      dq 0FFFFFFFFCA04h ; [1].monVPN
.monLoaderHeader:FFFFFFFFDE2F08C      dq 1             ; [1].monPages
.monLoaderHeader:FFFFFFFFDE2F094      dq 80000000000003h ; [1].flags
.monLoaderHeader:FFFFFFFFDE2F09C      dd 0             ; [1].allocs
.monLoaderHeader:FFFFFFFFDE2F0A0      db 4 dup(0)      ; 1
.monLoaderHeader:FFFFFFFFDE2F0A4      dq 0             ; [1].blobSrc.offset
.monLoaderHeader:FFFFFFFFDE2F0AC      dq 0             ; [1].blobSrc.size
.monLoaderHeader:FFFFFFFFDE2F0B4      dq 0             ; [1].bspOnly
.monLoaderHeader:FFFFFFFFDE2F0BC      dq 0             ; [1].subIndex
```



SharedArea Details

- **vmx is responsible for allocating memory and building page table structures based on vmmblob's information**
- **vmx86.sys further populates the page table information and loads the vmm ELF file constructed by vmx**
- **vmx, vmmblob, vmx86.sys work together to build the vmm's enviroment, mapping the host allocated address to vmm's virtual address**



VMM Switch Details

- **CrossPage: one page size memory region allocated for each vcpu by vmx86.sys**
- **Responsible for storing context between vmm and the host, like VMCS**
- **Mapped to the virtual page 0xFFFFFFFFFCA00 of vmm, according to .monloader section**

```
.monLoaderHeader:FFFFFFFFDE2F2BC ; const MonLoaderEntry
.monLoaderHeader:FFFFFFFFDE2F2BC      dd ML_CONTENT_SHARE      ; content
.monLoaderHeader:FFFFFFFFDE2F2C0      dd MonLoaderSourceType:ML_SOURCE_HOST; source
.monLoaderHeader:FFFFFFFFDE2F2C4      dq 0FFFFFFFFFCA00h        ; monVPN
.monLoaderHeader:FFFFFFFFDE2F2CC      dq 1                      ; monPages
.monLoaderHeader:FFFFFFFFDE2F2D4      dq 8000000000000003h      ; flags
.monLoaderHeader:FFFFFFFFDE2F2DC      dd 0                      ; allocs
.monLoaderHeader:FFFFFFFFDE2F2E0      db 4 dup(0)
.monLoaderHeader:FFFFFFFFDE2F2E4      dq 0                      ; blobSrc.offset
.monLoaderHeader:FFFFFFFFDE2F2EC      dq 0                      ; blobSrc.size
.monLoaderHeader:FFFFFFFFDE2F2F4      dq 0                      ; bspOnly
.monLoaderHeader:FFFFFFFFDE2F2FC      dq 9                      ; subIndex
```



VMM Switch Details

- **vmx uses IOCTL_VMX86_RUN_VM in main loop**
- **vmx86.sys calls the HostSwitchToVmm code snippet**
- **The host is responsible for saving the current CPU state to CrossPage, including system-level context such as the cr3 register**

```
HostSwitchToVmm:
; CODE XREF: Task_Switch+59C1p
push    qword ptr [rsp+8] ; save Host cs reg and call's return address to stack. then push them to CrossPage.
; VMM can use CrossPage and retfq back to Host
mov     word ptr [rsp+8], cs
mov     rax, rsp
lea     rsp, [rcx+162h]
mov     dx, ss
push    dx
push    rax
push    r15
push    r14
push    r13
push    r12
push    rdi
push    rsi
push    rbp
push    rbx
mov     rax, cr3      ; host cr3
push    rax
lea     rsp, [rcx+270h]
pop     rsi           ; root mpn
pop     rbx
pop     rbp
pop     r12
pop     r13
pop     r14
pop     r15
pop     rax
pop     dx
lgdt    fword ptr [rcx+338h] ; rcx point to crosspage
mov     cr3, rsi
mov     ds, edx
mov     es, edx
mov     ss, edx
mov     rsp, rax
retfq   ; Jmp [rsp], rsp += 8, mov cs, [rsp], rsp += 8
; -----
align 10h
```

Structures

00000263	db ? ; unde
00000264	db ? ; unde
00000265	db ? ; unde
00000266	db ? ; unde
00000267	db ? ; unde
00000268	db ? ; unde
00000269	db ? ; unde
0000026A	db ? ; unde
0000026B	db ? ; unde
0000026C	db ? ; unde
0000026D	db ? ; unde
0000026E	db ? ; unde
0000026F	db ? ; unde
00000270	page_table_root_mpn dq ?
00000270	
00000278	switch_to_vmm_rbx dq ?
00000280	switch_to_vmm_rbp dq ?
00000288	switch_to_vmm_r12 dq ?
00000290	switch_to_vmm_r13 dq ?
00000298	switch_to_vmm_r14 dq ?
000002A0	switch_to_vmm_r15 dq ?
000002A8	switch_to_vmm_rsp dq ?
000002B0	switch_to_vmm_dx dq ?
000002B8	CrossPagePhyMachineAddress
000002B8	
000002C0	db ? ; unde
000002C1	db ? ; unde
000002C2	db ? ; unde
000002C3	db ? ; unde
000002C4	db ? ; unde
000002C5	db ? ; unde



UserRpc In VMM

- **UserRpc** is implemented through **PlatformUserCall** in **vmm**
- Saves the opcode to the address **0xFFFFFFFFFCA00550**
- Place the **PlatformCall** invocation number **100** at **0xFFFFFFFFFCA00428**
- These addresses are actually offsets within **CrossPage**

```
int __fastcall PlatformUserCall(UserCallOperation op)
{
    int result; // eax

    MEMORY[0xFFFFFFFFFCA00550] = op;
    MEMORY[0xFFFFFFFFFCA00428] = 100;
    BackToHost();
    result = MEMORY[0xFFFFFFFFFCA0042C];
    MEMORY[0xFFFFFFFFFCA00550] = 300;
    return result;
}
```



UserRpc In VMM

- **PlatformCall 100 causes vmx86.sys to choose whether to return the opcode saved at CrossPage offset 0x550 to vmx**
- **In the main loop of IOCTL_VMX86_RUN_VM in vmx, vmx selects the corresponding UserRpcHandler based on this opcode number**
- **UserRpcBlock, it is precisely the content saved by vmm via SharedArea, in the direct memory mapping between the host and vmm memory**

```
V23 = V39;  
LODWORD(userRpcBlock[1]) = 65280;  
HIDWORD(userRpcBlock[1]) = v23;  
LODWORD(userRpcBlock[2]) = v40;  
HIDWORD(userRpcBlock[2]) = a4;  
userRpcBlock[4] = v43;  
UserRPC(334);
```



Device Virtualization

- vmm resolves the x86 IO instructions, and may attempt to use **UserRPC(317)** to call vmx to process

```
{
    v31 = 0;
    if ( a4 == 1 )
        v31 = *a8;
    }
    HIDWORD(userRpcBlock[7]) = v31;
}
UserRPC(317);
iospaceCurrentBA = 0LL;
if ( (a3 & 2) != 0 )
{
    v32 = userRpcBlock[1];
    switch ( a4 )
    {
        case 4:
            *(_DWORD *)a8 = v32;
            break;
        case 2:
            *(_WORD *)a8 = v32;
            break;
        case 1:
            *a8 = v32;
            break;
    }
}
}
```

```
for ( i = 0; v17 < v6; v17 += InputOutputSize * v19 )
{
    v19 = ((__int64 (__fastcall *)(IoUserCallback *, _QWORD, _QWORD, _QWORD, int, char *))ioPortCallbackFunction->ioPortCallback)(
        ioPortCallbackFunction->ExtArg,
        IoPort_,
        (unsigned int)(repTimes - i),
        (unsigned int)rpcBlock->InputOutputSize,
        rpcFlag,
        &v8[v17]);
}
```



Device Virtualization

- Some devices implement their IOCallback in vmm, not in vmx

```
.rodata:00000000000D1C80 iospaceCBs      dq offset Vmxnet3_IODataHandler
.rodata:00000000000D1C80                ; DATA XREF: MonC
.rodata:00000000000D1C88                dq offset BusMemBalloon_BackdoorPort
.rodata:00000000000D1C90                dq offset PortF0h_Handler
.rodata:00000000000D1C98                dq offset Port92h_Handler
.rodata:00000000000D1CA0                db      0
.rodata:00000000000D1CA1                db      0
.rodata:00000000000D1CA2                db      0
.rodata:00000000000D1CA3                db      0
.rodata:00000000000D1CA4                db      0
.rodata:00000000000D1CA5                db      0
.rodata:00000000000D1CA6                db      0
.rodata:00000000000D1CA7                db      0
.rodata:00000000000D1CA8                dq offset PIC_CmdPort
.rodata:00000000000D1CB0                dq offset PIC_MaskPort
.rodata:00000000000D1CB8                dq offset PIC_TriggerPort
.rodata:00000000000D1CC0                dq offset Backdoor_PortMon
.rodata:00000000000D1CC8                dq offset Vmxnet3_IODataHandler
.rodata:00000000000D1CD0                dq offset Vmxnet3_IODataHandler
.rodata:00000000000D1CD8                dq offset Vmxnet3_IODataHandler
.rodata:00000000000D1CE0                dq offset Vmxnet3_IODataHandler
.rodata:00000000000D1CE8                dq offset Vmxnet3_IODataHandler
.rodata:00000000000D1CF0                dq offset CMOS_AddrPort
.rodata:00000000000D1CF8                dq offset CMOS_ValPort
.rodata:00000000000D1D00                dq offset E1000_IoDataHandler
.rodata:00000000000D1D08                dq offset LSILogic_CommonIOHandler
.rodata:00000000000D1D10                dq offset Vmxnet3_IODataHandler
.rodata:00000000000D1D18                dq offset PCI_ConfData
```



Device Virtualization

- For memory-mapped I/O (MMIO), in most cases, vmx associates most memory regions with a specific ID, linking them to corresponding MemHandler functions in vmm by default

```
if ( (*(_DWORD *) (qword_7FF7CF10D180 + 416) & 0x100) == 0
|| (v3 = AllocMem(v2, 1u, (__int64)"ControlBar", 18, (__int64)sub_7FF7CE379640, v9, 0xA6u),
    (*(_DWORD *) (v0 + 144) = v3,
    v3 < 0x3CE) )
{
    ...
}
```

```
.rodata:00000000000D1B80 physMemIOCBs      dq offset Vmxnet3_MemHandler; 0
.rodata:00000000000D1B80                                ; DATA XREF: MonCB_GetPhysMemIOFunc+7
.rodata:00000000000D1B88      dq offset APIC_RegisterAccess; 1
.rodata:00000000000D1B90      dq offset IOAPIC_RegisterAccess; 2
.rodata:00000000000D1B98      dq offset PhysMem_IUserCallback; 3
.rodata:00000000000D1BA0      dq offset Vmxnet3_MemHandler; 4
.rodata:00000000000D1BA8      dq offset E1000_MemMapHandler; 5
.rodata:00000000000D1BB0      dq offset Ehci_MemMapHandler; 6
.rodata:00000000000D1BB8      dq offset HDAudio_MemMapHandler; 7
.rodata:00000000000D1BC0      dq offset HPET_MemHandler; 8
.rodata:00000000000D1BC8      dq offset LSILogic_MemoryMappedHandler; 9
.rodata:00000000000D1BD0      dq offset NVDIMM_FlushHandler; 10
.rodata:00000000000D1BD8      dq offset Vmxnet3_MemHandler; 11
.rodata:00000000000D1BE0      dq offset PCIE_MMIO             ; 12
.rodata:00000000000D1BE8      dq 5 dup(offset Vmxnet3_MemHandler); 13
.rodata:00000000000D1C10      dq offset SVGA_ControlBarMemRef; 18
.rodata:00000000000D1C18      dq offset SVGA_RegsBarMemRef; 19
.rodata:00000000000D1C20      dq offset VMCI_RegMemHandler; 20
.rodata:00000000000D1C28      dq offset VGA_MemRef             ; 21
.rodata:00000000000D1C30      dq 8 dup(offset Vmxnet3_MemHandler); 22
.rodata:00000000000D1C70      dq offset Xhci_MemMapHandler; 30
```



Device Virtualization

- **Most MMIO will access the SharedArea in vmm to interact with vmx**
- **Most MMIO operations ultimately still rely on UserRpc to call the relevant processing routines in vmx**
- **To understand the specific UserRpcBlock content, you should check the corresponding UserRpc call in vmm**



Device Virtualization

- Almost all methods for accessing the physical memory follow a similar calling pattern
- The representation object of physical memory is obtained based on the physical address
- Depending on the object's type, direct memory access within vmx is usually used

```
if ( PhysMem_ValidateAndGet(phyAddr, pageSize, 1u, 9u, &PhyMemContent) )
{
    LODWORD(RingPointerPA) = ConsumerRing->RingPointerPA;
    enqueuePtr = ConsumerRing->enqueuePtr;
    while ( 1 )
    {
        v10 = (unsigned int)(RingPointerPA - phyAddr);
        v11 = &ConsumerRing->TrbRingQueue[enqueuePtr];
        v12 = (unsigned int)(v10 + 12);
        if ( PhyMemContent.type == 1 )
        {
            v13 = *(_DWORD *)(&PhyMemContent.contentHostVA + v12);
        }
        else
        {
            PhysMemReadSlow(&PhyMemContent, v12, 4ui64, (char *)&v22);
            v13 = v22;
        }
    }
}
```



Device Virtualization

- In ESXi, some devices' MMIO will not always call UserRPC
- Some devices will call UserRPC only when “hostedEmulation” is enabled
- vmkcall - vmm direct call to VMKernel to handle devices emulation

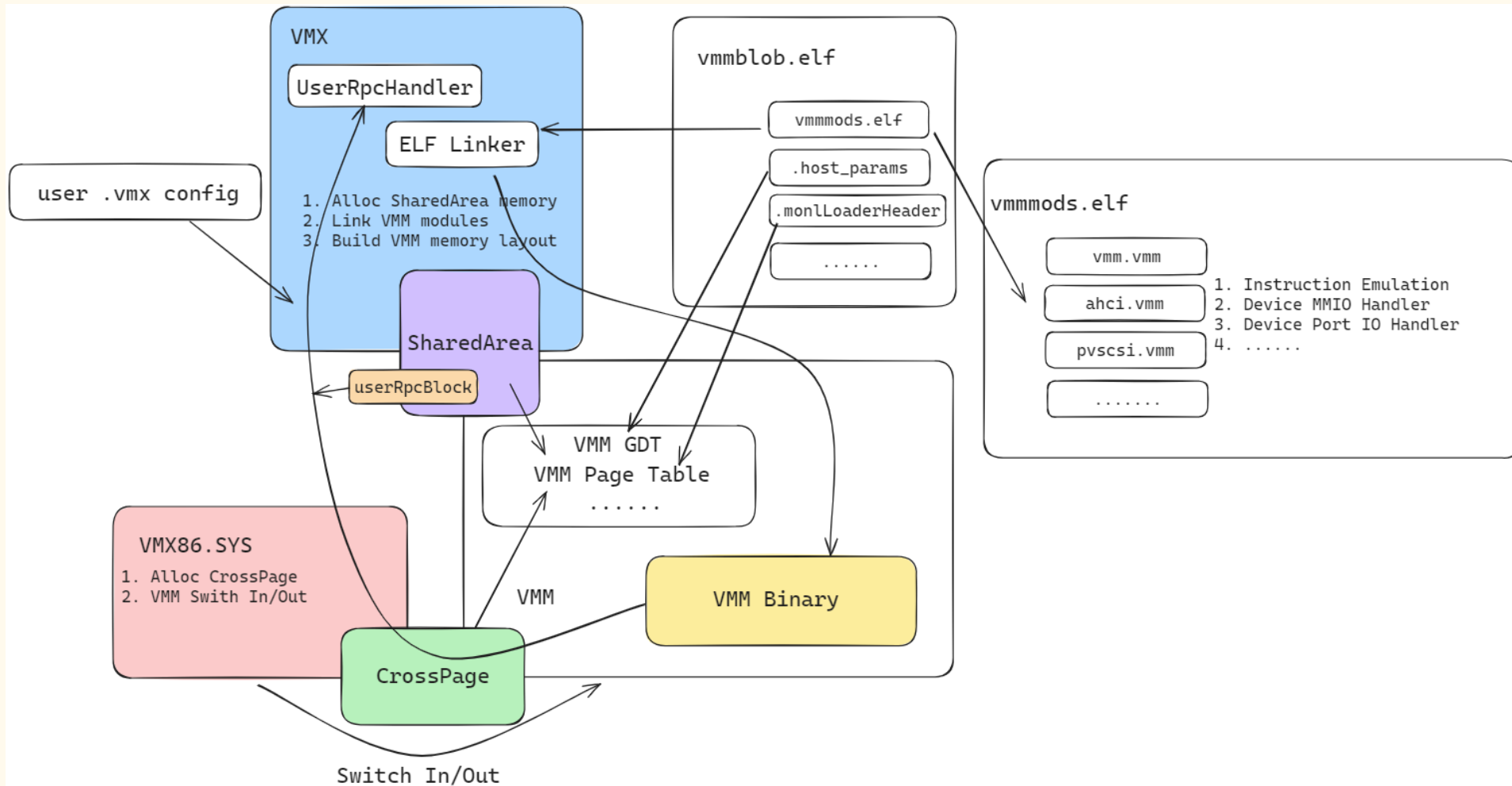
```
__int64 __fastcall Platform_PVSCSIProcessRing(unsigned int a1)
{
    return VMK_Call_1Args(133LL, a1);    // call vmkfunc 133
}
```

```
.rodata:00004200007D3AB0      dq offset Net_VMMVlanceUpdateMAC; 118
.rodata:00004200007D3AB8      dq offset Net_VMMVmxnetUpdateEthFRP; 119
.rodata:00004200007D3AC0      dq offset VSCSI_ExecuteCommand; 120
.rodata:00004200007D3AC8      dq offset VSCSI_CmdComplete; 121
.rodata:00004200007D3AD0      dq offset VSCSI_AccumulateSG; 122
.rodata:00004200007D3AD8      dq offset VSCSI_FreeSG ; 123
.rodata:00004200007D3AE0      dq offset VSCSI_MapMPN ; 124
.rodata:00004200007D3AE8      dq offset LSI_InitRings ; 125
.rodata:00004200007D3AF0      dq offset LSI_ProcessReq; 126
.rodata:00004200007D3AF8      dq offset LSI_ActivatePoll; 127
.rodata:00004200007D3B00      dq offset LSI_ProcessCompl; 128
.rodata:00004200007D3B08      dq offset VSCSI_ChangeCompletionMode; 129
.rodata:00004200007D3B10      dq offset PVSCSI_AdapterInit; 130
.rodata:00004200007D3B18      dq offset PVSCSI_FlushIotlb; 131
.rodata:00004200007D3B20      dq offset PVSCSI_SyncCmd; 132
.rodata:00004200007D3B28      dq offset PVSCSI_ProcessRing; 133
.rodata:00004200007D3B30      dq offset PVSCSI_PromoteCompletions; 134
.rodata:00004200007D3B38      dq offset PVSCSI_ProcessCompletion; 135
.rodata:00004200007D3B40      dq offset PVSCSI_DisableReqCallCoalescing; 136
.rodata:00004200007D3B48      dq offset PVSCSI_EnableReqCallCoalescing; 137
.rodata:00004200007D3B50      dq offset PVSCSI_DisableAsyncProcessing; 138
.rodata:00004200007D3B58      dq offset PVSCSI_EnableAsyncProcessing; 139
.rodata:00004200007D3B60      dq offset PVSCSI_CheckShadowRingQuiesced; 140
.rodata:00004200007D3B68      dq offset Net_VMMStopPacketFilter; 141
.rodata:00004200007D3B70      dq offset VMKPCIPassthru_UnmaskVector; 142
.rodata:00004200007D3B78      dq offset VMKPCIPassthru_UpdatePrivateDomain; 143
.rodata:00004200007D3B80      dq offset VMKPCIPassthru_SetAddressDomain; 144
```

vmkFuncTable



VMware Hypervisor



Device Virtualization

When you reverse engineer the device emulation code in vmx ...

- **Not sure where the device's data is coming from?**

Go check vmm code

- **Not sure how to trigger the device's code?**

Go check vmm code

- **Want to find some a new device's code?**

Go check vmm code



VMware Device Virtualization Bug Hunting



USB Emulation Architecture

- **USB Host Controller Emulation**

UHCI, EHCI, XHCI

- **VUSB Emulation**

Urb Object, Pipe Object, Port Object ...

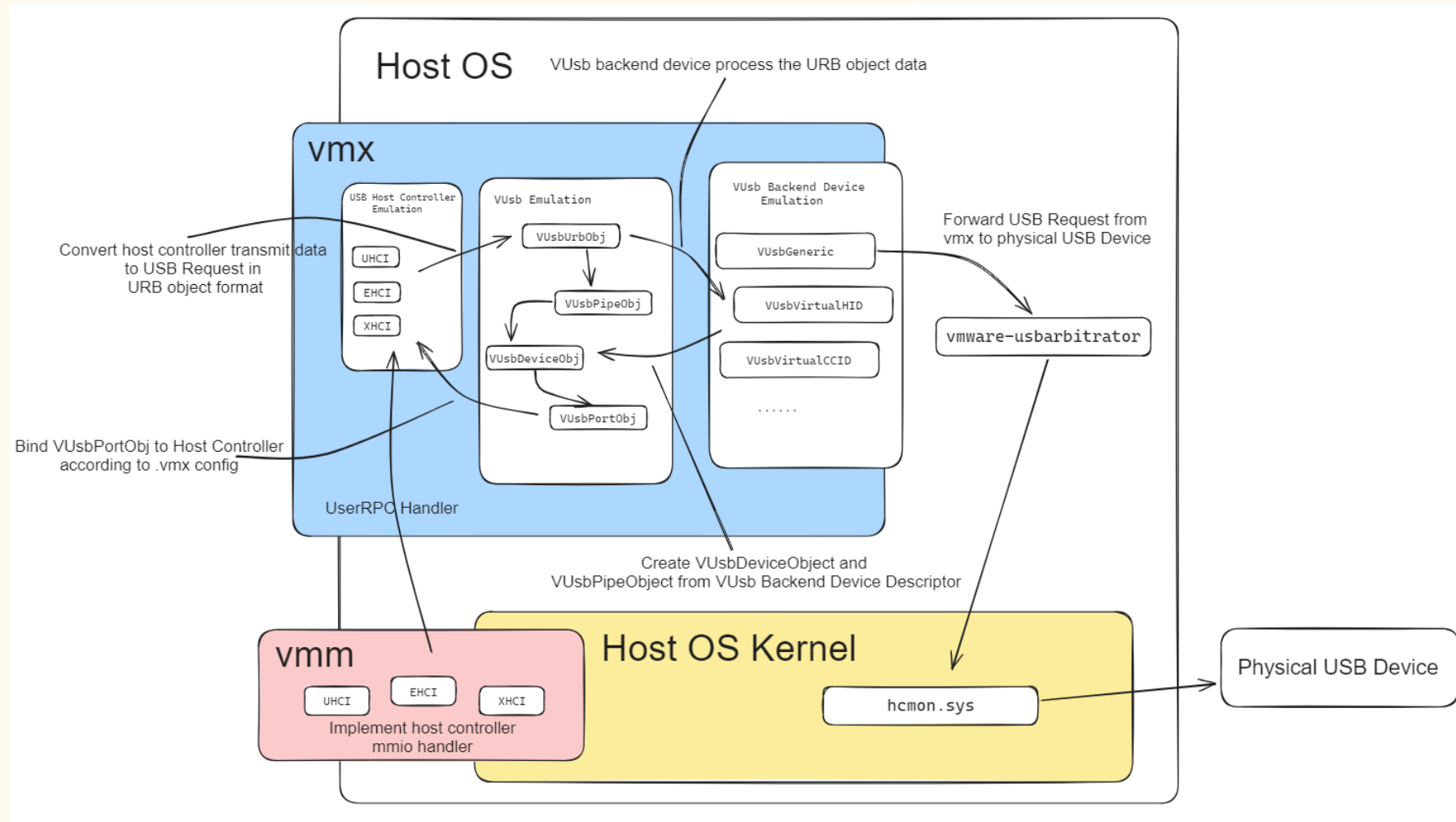
- **VUSB Backend Device Emulation**

Generic, Bluetooth, Rng ...

```
; VusbBackendInitializeEntry VusbBackendInitializeTable
VusbBackendInitializeTable VusbBackendInitializeEntry <offset VusbVirtualHubBackendClass, \
; DATA XREF: Vusb_PowerOn+39F10
; offset VirtualHubDeviceRegisterVusbDeviceObject>
VusbBackendInitializeEntry <offset VusbGenericBackendClass, \
; offset sub_7FF7CE5667B0>
VusbBackendInitializeEntry <offset VusbVirtualHIDBackendClass, \
; offset VirtualHIDDeviceRegisterVusbDeviceObject>
VusbBackendInitializeEntry <offset VusbRemoteDeviceBackendClass, \
; offset sub_7FF7CE4E37C0>
VusbBackendInitializeEntry <offset VusbVirtualMassStorageBackendClass, \
; offset sub_7FF7CE5690B0>
VusbBackendInitializeEntry <offset VusbVirtualRNGBackendClass, \
; offset sub_7FF7CE5667F0>
VusbBackendInitializeEntry <offset VusbVirtualCCIDBackendClass, \
; offset sub_7FF7CE5667F0>
VusbBackendInitializeEntry <offset VusbVirtualBluetoothBackendClass, \
; offset sub_7FF7CE5667F0>
VusbBackendInitializeEntry <offset VusbVirtualAudioBackendClass, \
; offset sub_7FF7CE5667F0>
VusbBackendInitializeEntry <offset VusbVirtualVideoBackendClass, \
; offset sub_7FF7CE5694B0>
```



USB Emulation Architecture



CVE-2024-22255 - Uninitialized Memory



CVE-2024-22255

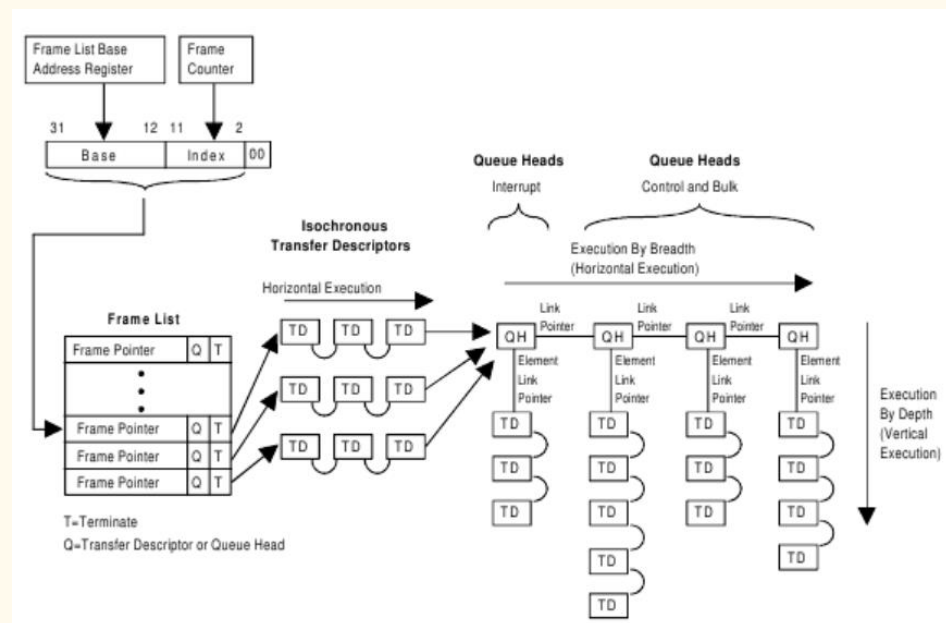
- **One of the payloads used by USB devices is the Standard Device Request, which begins in the format of Setup Packet**
- **One of the most interesting fields is wLength, which indicates the length of data requested to the USB device**

Offset	Size	Field
0	1	bmRequestType
1	1	bRequest
2	2	wValue
4	2	wIndex
6	2	wLength



CVE-2024-22255

- **The Standard Device Request serves as the payload for USB devices, but USB host controllers do not transfer data based on this unit**
- **For UHCI, data is transferred in units of Transfer Descriptors (TDs) and linked in guest memory in a list-like structure known as Queue Head (QH)**



CVE-2024-22255

- **When processing control transfers, VMware's UHCI controller allocates URB objects on a per-Standard Device Request basis**
- **VMware retrieves the first TD on the Queue Head (QH) and uses it as the starting point to parse the Setup Packet**
- **It extracts the wLength field from the Setup Packet and adds the size of the Setup Packet to determine the size of the data buffer for the URB object**



CVE-2024-22255

```
urbBufferSize = tdDataSize + HIWORD(v27); // wLength
URB = VUsb_AllocUrb(vusbPipeObject, 0i64, (unsigned int)urbBufferSize); // alloc urb with size sizeof(setup packet) + wLength
URB->urbAllocatedDataBufferSize = urbBufferSize;

LABEL_55:
if ( !URB )
{
LABEL_44:
Log_Level(0x1E43u, "UHCI: Unexpected in/out %d pid %2x.\n", (unsigned int)v7, *((unsigned __int8 *)v10 + 24));
v10[5] &= 0xE7FFFFFFF;
v10[5] |= 0x6407FFu;
v19 = *(unsigned int **)( (_QWORD *)v10 + 8i64);
v20 = *v19;
v19[1] = v10[5];
if ( (_DWORD)v20 != v10[4] )
{
    LODWORD(v21) = v10[4];
    Log_Level(
        0x1E43u,
        "UHCI: Link pointer in TD changed: TD %#I64x, new link %x, old link %x.\n",
        **(_QWORD **)v10,
        v20,
        v21);
}
*( _DWORD *) (a1 + 1640) |= 5u;
LABEL_47:
v18 = v22;
goto LABEL_48;
}
_mm_lfence();
if ( (int)tdDataSize > urbBufferSize ) // check tdsiz not overflow the urbBufferSize
{
    _mm_lfence();
    Log_Level(0x1E43u, "UHCI: Setup data packet over run %d %d.\n", tdDataSize, (unsigned int)urbBufferSize);
    tdDataSize = 0;
    if ( urbBufferSize > 0 )
        tdDataSize = urbBufferSize;
}
UrbDataBufferPointer = (void *)URB->UrbDataBufferPointer;
if ( v28.type == 1 )
{
    memcpy(UrbDataBufferPointer, (const void *)v28.contentHostVA, (int)tdDataSize);
}
else
{
    _mm_lfence();
    PhysMemReadSlow(&v28, 0i64, (int)tdDataSize, (char *)UrbDataBufferPointer);
}
URB->UrbDataBufferPointer += (int)tdDataSize;
goto LABEL_15;
}
```



CVE-2024-22255

- The allocation process of URB depends on the target device you are transferring to
- Different types of backend USB devices will result in URB objects with varying private structures

```
if ( bufferSize > v11 )
    Panic("UsbDev: URB greater than the max allowed URB size.\n");
_mm_lfence();
v12 = (VusbUrbObj *)((__int64 (__fastcall *) (VusbBackendDeviceObj *, _QWORD, _QWORD))pipeObject->VusbBackendDeviceObj->backendObj->VusbBackendUrbOperation->AllocUrb)(
    pipeObject->VusbBackendDeviceObj,
    (unsigned int)packets,
    bufferSize);
v12->UrbHandleReturnState = -1;
v12->IntervalEntry = (UrbIntervalEntry *)&v12[1];
v12->UrbDataBufferPointer = (__int64)v12->UrbDataBufferAllocedByUrbSize;
v12->StreamID = 0;
v12->vusbPipeObject = pipeObject;
*(__QWORD *)&v12->field_50 = 0i64;
v12->UrbSize = bufferSize;
*(__QWORD *)&v12->urbAllocatedDataBufferSize = 0i64;
v12->UrbFlowState = 0;
v12->RefCnt = 1;
v12->PipeType = pipeObject->PipeType;
v12->endPointAddr = pipeObject->endPointAddress;
backendObj = pipeObject->VusbBackendDeviceObj->backendObj;
v12->PipeUrbNode.front = &v12->PipeUrbNode;
v12->PipeUrbNode.next = &v12->PipeUrbNode;
v12->SubmitUrbNode.front = &v12->SubmitUrbNode;
v12->SubmitUrbNode.next = &v12->SubmitUrbNode;
v12->backendObj = backendObj;
v12->field_68 = 0;
v12->PacketQueueHelper = 0i64;
```



CVE-2024-22255

- **For HID devices, when allocating URB objects, no additional structures are added besides the generic data fields of the URB**
- **Additionally, HID devices utilize malloc for data allocation**

```
VusbUrbObj *__fastcall UsbVirtualHIDAllocUrb(__int64 a1, unsigned int a2, unsigned int a3)
{
    __int64 v3; // rbx
    VusbUrbObj *urb; // rax

    v3 = 12i64 * a2;
    urb = (VusbUrbObj *)Util_SafeMalloc(v3 + a3 + 0x98i64);
    urb->GenericDeviceUrbPrivateField = (GenericDeviceUrbPrivateFieldObj *)&unk_7FF7CF66AB30;
    urb->UrbDataBufferAllocatedByUrbSize = (char *)&urb[1] + v3;
    return urb;
}
```



CVE-2024-22255

- Allocating wLength sized URB doesn't mean you will get wLength sized data from guest supplied TDs
- Malloc allocation left memory uninitialized
- Backend USB device returns data through the same URB buffer, leading to a heap data leak

```
goto LABEL_26;
case USB_DEVICE_REQUEST_TYPE_SET_CONFIGURATION:
    if ( v7 < 0 || (v7 & 0x1F) != 1 )
        goto LABEL_26;
    v9 = *(void (fastcall *)(_QWORD, _QWORD, _QWORD, _BYTE *, int))(VUsbHIDDevice->VUsbHIDOperation + 40);
    if ( v9 )
        v9(
            *((unsigned __int16 *)UrbDataBufferAllocedByUrbSize + 2),
            HIBYTE(*((unsigned __int16 *)UrbDataBufferAllocedByUrbSize + 1)),
            (unsigned __int8)*((_WORD *)UrbDataBufferAllocedByUrbSize + 1),
            v8,
            urb->urbAllocatedDataBufferSize - 8);
    break;
case USB_DEVICE_REQUEST_TYPE_GET_INTERFACE:
    urb->UrbHandleReturnState = 0;
    urb->UrbReturnDataSize = 8;
    goto LABEL_34;
default:
LABEL_26:
    urb->UrbHandleReturnState = 3;
    goto LABEL_34;
}
if ( payloadSize >= 0 )
{
    urb->UrbHandleReturnState = 0;
    urb->UrbReturnDataSize = payloadSize + 8;
    goto LABEL_34;
}
goto LABEL_26;
}
```

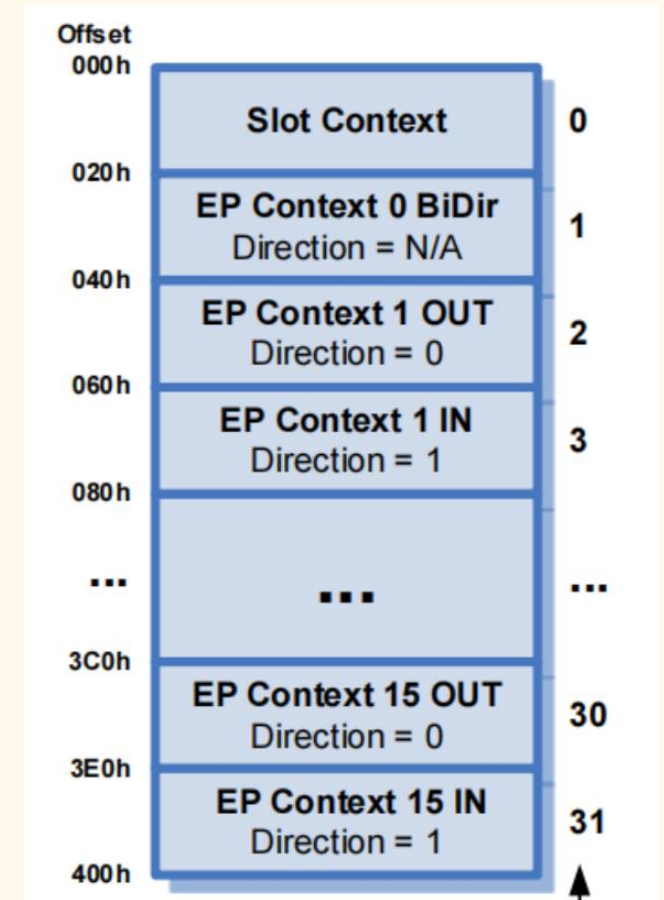


CVE-2024-22252 - Use After Free



CVE-2024-22252

- **Device Slot Context**
Element 0 points to a Slot Context structure, which holds information for the device
- **Endpoint Context**
An Endpoint Context structure holds context information for a single endpoint
- **Transfer Ring**
Each endpoint has one or more Transfer Rings. A Transfer Ring is an array of Transfer Request Blocks (TRBs)



CVE-2024-22252

- Look back to the old bug
- CVE-2021-22040
- Very interesting patch
- Before you figure out the XHCI emulation code, you may be confused

```
while ( v30 )
{
    _BitScanForward(&v18, v30);
    v19 = 1 << v18;
    v30 ^= 1 << v18;
    if ( v18 == -1 )
        break;
    v20 = 8i64 * (int)v18;
    v21 = *(_OWORD *)&v31[v20 + 4];
    *(_OWORD *)&v9[v20 + 4] = *(_OWORD *)&v31[v20];
    *(_OWORD *)&v9[v20 + 8] = v21;
    v9[2] |= v19;
    XhciStreams_FreeEndpoint(a1, v6, v18); // Bug! free after the context modification
    v22 = a1 + 1296i64 * v6;
    v23 = 32i64 * v18;
    v24 = *(_QWORD *)(v23 + v22 + 332536);
    if ( (v24 & 7) != 1 )
    {
        *(_QWORD *)(v23 + v22 + 332536) = v24 & 0xFFFFFFFFFFFFFFFF8ui64 | 1;
        *(_DWORD *)(v22 + 332528) |= v19;
    }
}

while ( v30 )
{
    _BitScanForward(&v18, v30);
    v19 = 1 << v18;
    v30 ^= 1 << v18;
    if ( v18 == -1 )
        break;
    XHCI_FreeEndpoint(a1, v6, v18); // patch, call free before the context modification
    v20 = 8i64 * (int)v18;
    v21 = 32i64 * v18;
    v22 = *(_OWORD *)&v31[v20 + 4];
    *(_OWORD *)(v20 * 4 + v9 + 16) = *(_OWORD *)&v31[v20];
    *(_OWORD *)(v20 * 4 + v9 + 32) = v22;
    *(_DWORD *)(v9 + 8) |= v19;
    v23 = a1 + 1296i64 * v6;
    v24 = *(_QWORD *)(v21 + v23 + 332536);
    if ( (v24 & 7) != 1 )
    {
        *(_QWORD *)(v21 + v23 + 332536) = v24 & 0xFFFFFFFFFFFFFFFF8ui64 | 1;
        *(_DWORD *)(v23 + 332528) |= v19;
    }
}
```

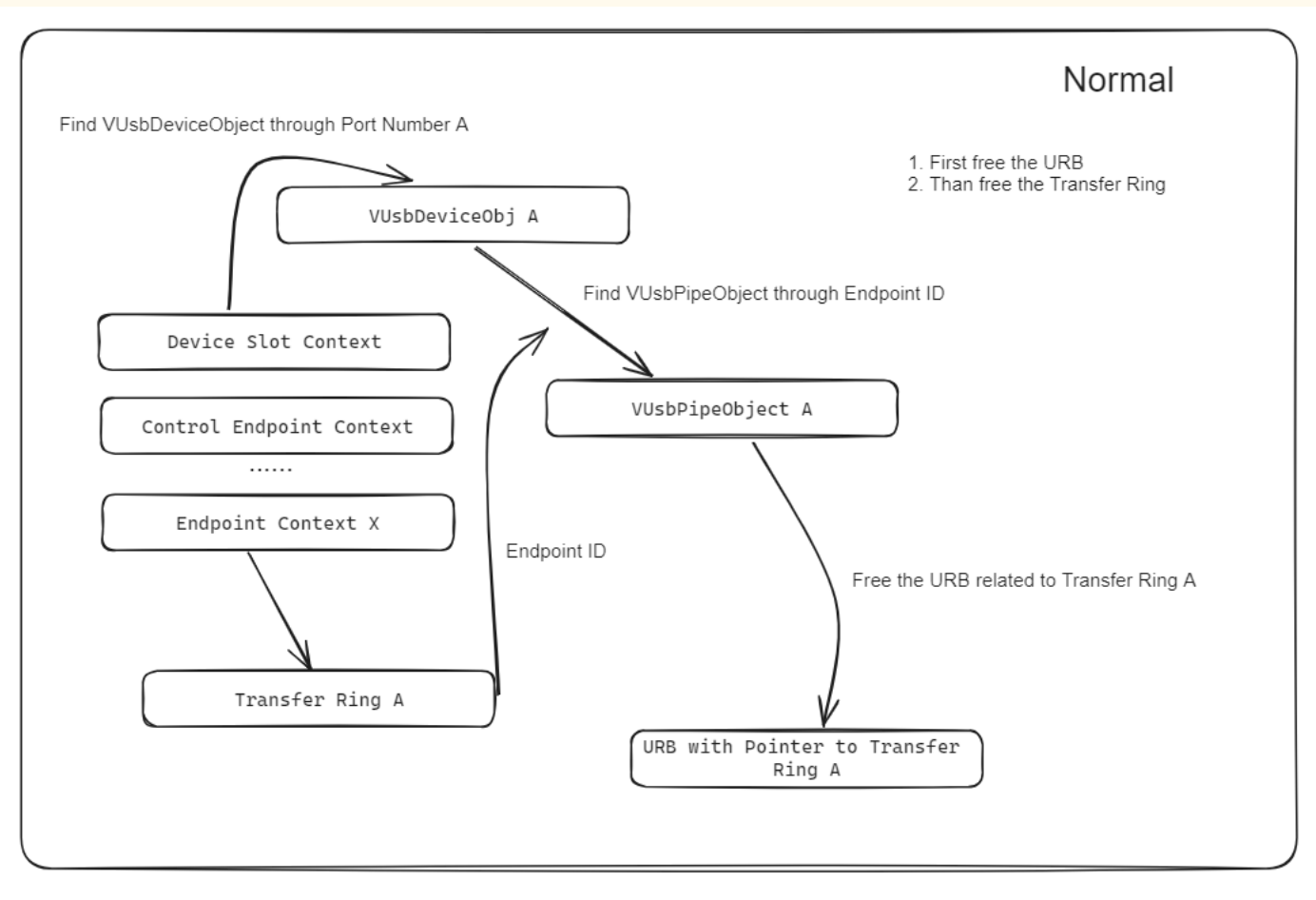


CVE-2024-22252

- **Get Transfer Ring Object from Transfer Ring hash table associated with this Endpoint Context**
- **Retrieves the VUsbDeviceObject based on the USB Port Number saved in the Device Slot Context**
- **Obtains the VUsbPipeObject from that VUsbDeviceObject based on the Endpoint ID**
- **Release URB Objects from that VUsbPipeObject**
- **Release Transfer Ring object**



CVE-2024-22252



CVE-2024-22252

```
Log_Level(
    6u,
    "UsbDev: DevID(%I64x): Cancel pipe(%p).\n",
    pipeObject->VUsbBackendDeviceObj->UsbDeviceProperties.DevID,
    pipeObject);
((void (__fastcall *)(VUsbBackendDeviceObj *, _QWORD))pipeObject->VUsbBackendDeviceObj->backendObj->VUsbBackendUrbOperation->CancelEndpoint)(
    pipeObject->VUsbBackendDeviceObj,
    (unsigned int)pipeObject->endPointAddress);
front = (__int64)pipeObject->URBList.front;
result = 0i64;
if ( (UrbListNode *)front != &pipeObject->URBList )
{
    // Release Urb on Pipe
    do
    {
        v4 = *(VUsb_PipeObject **)(front - 0x10);
        v5 = (VusbUrbObj *)(front - 40);
        v6 = *(VUsb_PipeObject **)(front);
        v7 = *(_DWORD *)(front - 40 + 0x50);
        urbAllocatedDataBufferSize = *(_DWORD *)(front - 40 + 8);
        LODWORD(v14) = v4->endPointAddress;
        Log_Level(
            7u,
            "UsbDev: DevID(%I64x): Removing URB(%p) from pipe(%p), endpt(%x).\n",
            v4->VUsbBackendDeviceObj->UsbDeviceProperties.DevID,
            (const void *)(front - 40),
            v4,
            v14);
    }
```



CVE-2024-22252

- **Endpoint Context is not the only object that holds a Transfer Ring Object pointer**
- **URB Object also holds a pointer that points to a field for Transfer Ring Object**
- **This field is responsible for tacking the corresponding TRB's data on Transfer Ring Object when XHCI returns USB device responses to the Guest**



CVE-2024-22252

- Before patch, XHCI commands like 'Configure Endpoint' could modify the contents of the Endpoint Context before releasing the Transfer Ring
- 'Configure Endpoint' could modify the contents of the Endpoint Context, leading the type mismatch with the VUsbPipeObject object type

```
if ( v30 )
{
    PipeType = (unsigned int)v30->PipeType;
    epType = (*(__QWORD *)packetQueue->endpointContext >> 35) & 7i64;
    if ( (_DWORD)PipeType == endpointType2PipeType[epType] )
    {
        // check endpoint type match pipe type
        return 1;
    }
    else
    {
        LODWORD(v34) = endPoint_;
        LODWORD(v33) = deviceSlot;
        Log_Level(
            0x1F03u,
            "XHCIPQUEUE: ERROR, mismatched pipe type (%d) for endpoint type (%d) on %02x:%02x, addr %d.\n",
            PipeType,
            epType,
            v33,
            v34,
            LOBYTE(v6->SlotContext[0].field4));
        result = 0;
        packetQueue->vusbPipeObject = 0i64; // modify the endpoint context made the vusbPipeObject be NULL!
    }
}
```



CVE-2024-22252

- **Left URB Object not freed, but related Transfer Ring Object already freed**
- **Dangling pointer - Use After Free**

```
if ( transferRing )
{
    XhciPacketQueue_Init(&v9, controller, transferRing->doorbellArg, &transferRing->packetQueueHelper); // didn't check whether we get the VUsbPipeObject!
    XhciPacketQueue_Cancel(&v9);
}
```

```
1 PacketQueueHelper *__fastcall XhciPacketQueue_Cancel(XHC_PacketQueue *packetQueue)
2 {
3     VUsb_PipeObject *vusbPipeObject; // rcx
4     PacketQueueHelper *result; // rax
5
6     vusbPipeObject = packetQueue->vusbPipeObject;
7     if ( vusbPipeObject ) // null, will not free the URB!
8     {
9         VUsb_CancelPipe(vusbPipeObject);
10        packetQueue->packetQueueHelper->TransferUrbLength = 0;
11        result = packetQueue->packetQueueHelper;
12        result->UrbField = 0;
13    }
14    return result;
15 }
```



CVE-2024-22252

- It is still possible to modify the **Device Slot Context** to retrieve another **VUsbDeviceObject**, leading to the inability to obtain the correct **VUsbPipeObject**

Slot Context Data Structure

31	27	26	25	24	23	22	21	20	19	18	17	16	15	8	7	0			
Context Entries				Hub	MTT	RsvdZ	Speed			Route String							03-00H		
Number of Ports						Root Hub Port Number						Max Exit Latency					07-04H		
Interrupter Target								RsvdZ		TTT		TT Port Number				TT Hub Slot ID			0B-08H
Slot State			RsvdZ											USB Device Address				0F-0CH	
xHCI Reserved (RsvdO)																13-10H			
xHCI Reserved (RsvdO)																17-14H			
xHCI Reserved (RsvdO)																1B-18H			
xHCI Reserved (RsvdO)																1F-1CH			



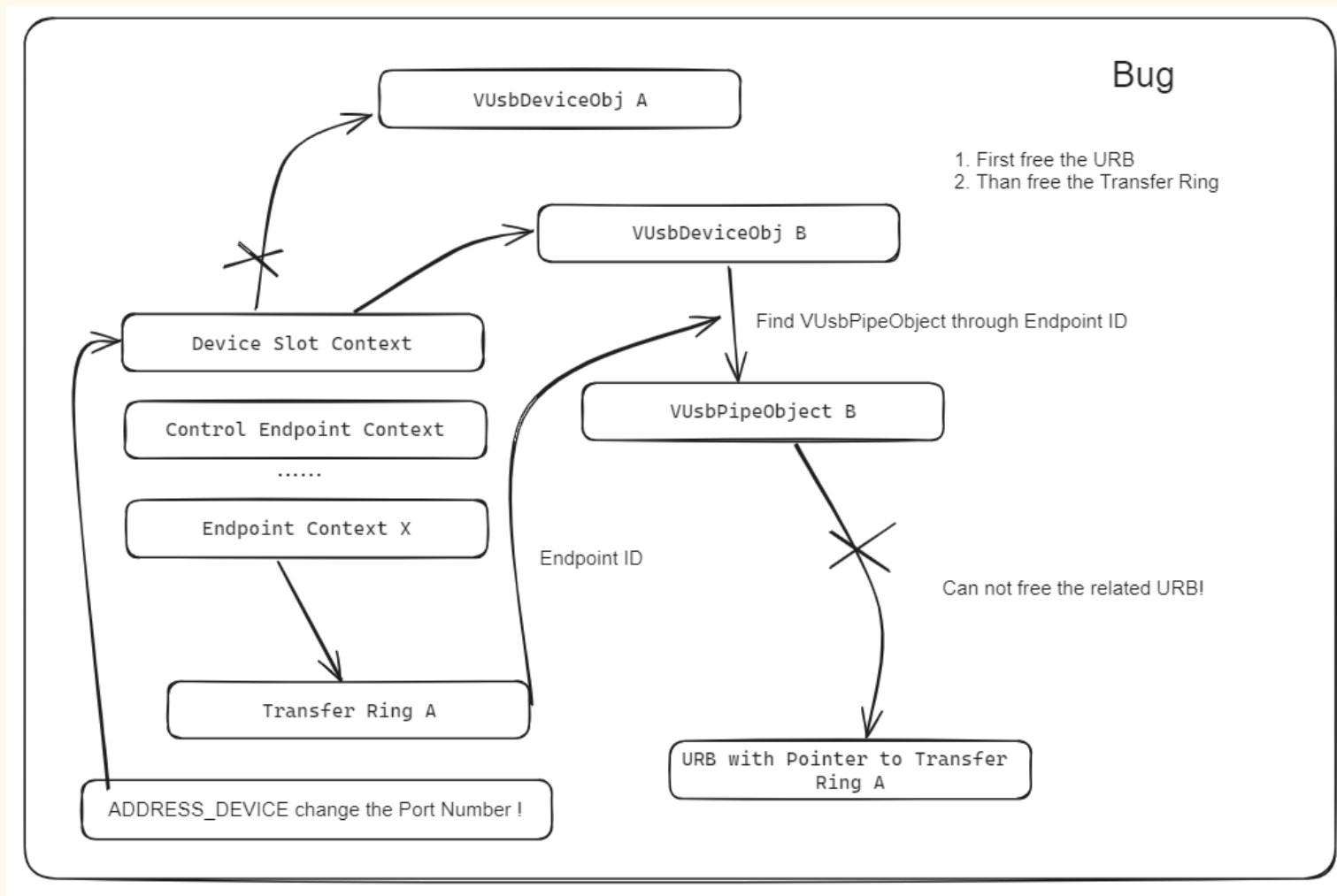
CVE-2024-22252

- **First complete the configuration process for a device, and create Transfer Rings on non-Control Endpoints**
- **Transfer URB data on those Transfer Rings**
- **Use the 'ADDRESS_DEVICE' command on that Device Slot to modify the Device Port Number in the Slot Context to point to another USB device**
- **VMware's implementation ensures that 'ADDRESS_DEVICE' does not affect other non-Control Endpoint Contexts**

```
XhciStreams_FreeEndpoint(controller, slotId_Minus_1, 1i64);
xhc_device_context->SlotContext[0] = XHC_InputContext.SlotContext[0];
*(_OWORD *)&xhc_device_context->ControlEndpoint0.field1 = *(_OWORD *)&XHC_InputContext.EndpointContexts[0].field1;
v9 = *(_OWORD *)&XHC_InputContext.EndpointContexts[0].field5; // ADDRESS_DEVICE Only Modify the Slot Context and Control Endpoint Context
xhc_device_context->field_8 = -1;
*(_OWORD *)&xhc_device_context->ControlEndpoint0.field5 = v9;
sub_7FF7CE575D50(controller, slotId_Minus_1, 1u, 1);
v10 = XHC_InputContext.SlotContext[0].field1 & 0xFFFFF;
v11 = (unsigned int)BYTE2(XHC_InputContext.SlotContext[0].field2) - 1;
if ( (unsigned int)v11 < controller->usbPortInformation.numMaxPorts )
{
```



CVE-2024-22252



CVE-2024-22251 - Out-of-Bounds Read



CVE-2024-22251

- **The Guest OS communicates with SmartCard through the Virtual SmartCard Reader**
- **Guest OS use CCID protocol to communicate with Virtual SmartCard Reader**
- **The APDU (Application Protocol Data Unit) serves as the data unit for interaction between the SmartCard Reader and the SmartCard**



CVE-2024-22251

```
00000000 ccid_xfrblock_msg_hdr struc ; (sizeof=0xA, mappedto_759)
00000000                                     ; XREF: ccid_xfrbl
00000000 msg_type          db ?
00000001 msg_len          dd ?
00000005 slot_num        db ?
00000006 seq_num         db ?
00000007 bwi             db ?
00000008 level_param     dw ?
0000000A ccid_xfrblock_msg_hdr ends

00000000 command_apdu      struc ; (sizeof=0x5, mappedto_760)
00000000                                     ; XREF: ccid_xfrbl
00000000 cla             db ?
00000001 ins             db ?
00000002 p1              db ?
00000003 p2              db ?
00000004 len             db ?
00000005 command_apdu    ends

00000000 ccid_xfrblock_msg_with_command_apdu struc ; (sizeof=0xF, m
00000000 hdr              ccid_xfrblock_msg_hdr ?
0000000A apdu             command_apdu ?
0000000F ccid_xfrblock_msg_with_command_apdu ends
```



CVE-2024-22251

- VMware checks whether the 'msg_len' field of ccid_xfrblock_msg_hdr matches the 'len' field of the command_apdu
- However, it fails to verify whether these two fields conform to the size of the URB buffer

```
msg_len = Buffer->hdr.msg_len;
APDU_LEN = msg_len - 4;
if ( msg_len < 4 )
{
    LogInfo("USB-CCID: Invalid len of APDU.\n", APDU_LEN); // Application Protocol Data Unit
    v8 = 0;
LABEL_41:
    v16 = (char *)Util_SafeCalloc(1ui64, 0xAui64);
    goto LABEL_42;
}
if ( (unsigned int)APDU_LEN >= 2 )
{
    // only check the apdu len match the msg_len
    // but what about URB data buffer?

    len = (unsigned __int8)Buffer->apdu.len;
    if ( ((_DWORD)APDU_LEN != len + 1 || !(_BYTE)len) && ((_DWORD)APDU_LEN != len + 2 || !(_BYTE)len) )
    {
        LogInfo(
            "USB-CCID: Unexpected apdu case, CLA:0x%1x, INS:0x%1x, P1:0x%1x, P2:0x%1x.\n",
            (unsigned __int8)Buffer->apdu.cla,
            (unsigned __int8)Buffer->apdu.ins,
            (unsigned __int8)Buffer->apdu.p1,
            (unsigned __int8)Buffer->apdu.p2);
        v8 = 0;
        goto LABEL_41;
    }
}
```



CVE-2024-22251

- **Directly uses these fields as parameters to call the Windows SCardTransmit API**
- **SCardTransmit takes a buffer pointer and buffer size as parameters and cannot verify the validity between these two parameters**
- **Out-of Bounds Access to Heap Data**

```
} // OOB Read occurs in SCardTransmit!  
v20 = SCardTransmit(  
    ccidDevice->hCard,  
    v19,  
    (LPCBYTE)&Buffer->apdu,  
    Buffer->hdr.msg_len,  
    0i64,  
    (LPBYTE)v16 + 10,  
    &pcbRecvLength);  
v21 = v20;  
...
```



Conclusion

- **Host controller emulation can be attacked**
- **VUSB emulation can be attacked**
- **USB device emulation can be attacked**
- **We have other cases we did not include in this presentation, but you can differ the vmx binary to found.**
- **More attack scenarios in the future?**
 - **Plug in an evil USB device and leverage vmx (Generic USB device, ...) to execute code?**
 - **Leverage local USB service (usbarbitrator, ...) to privilege escalation?**
 - **.....**
- **Very challenging to defend such a complex system.**

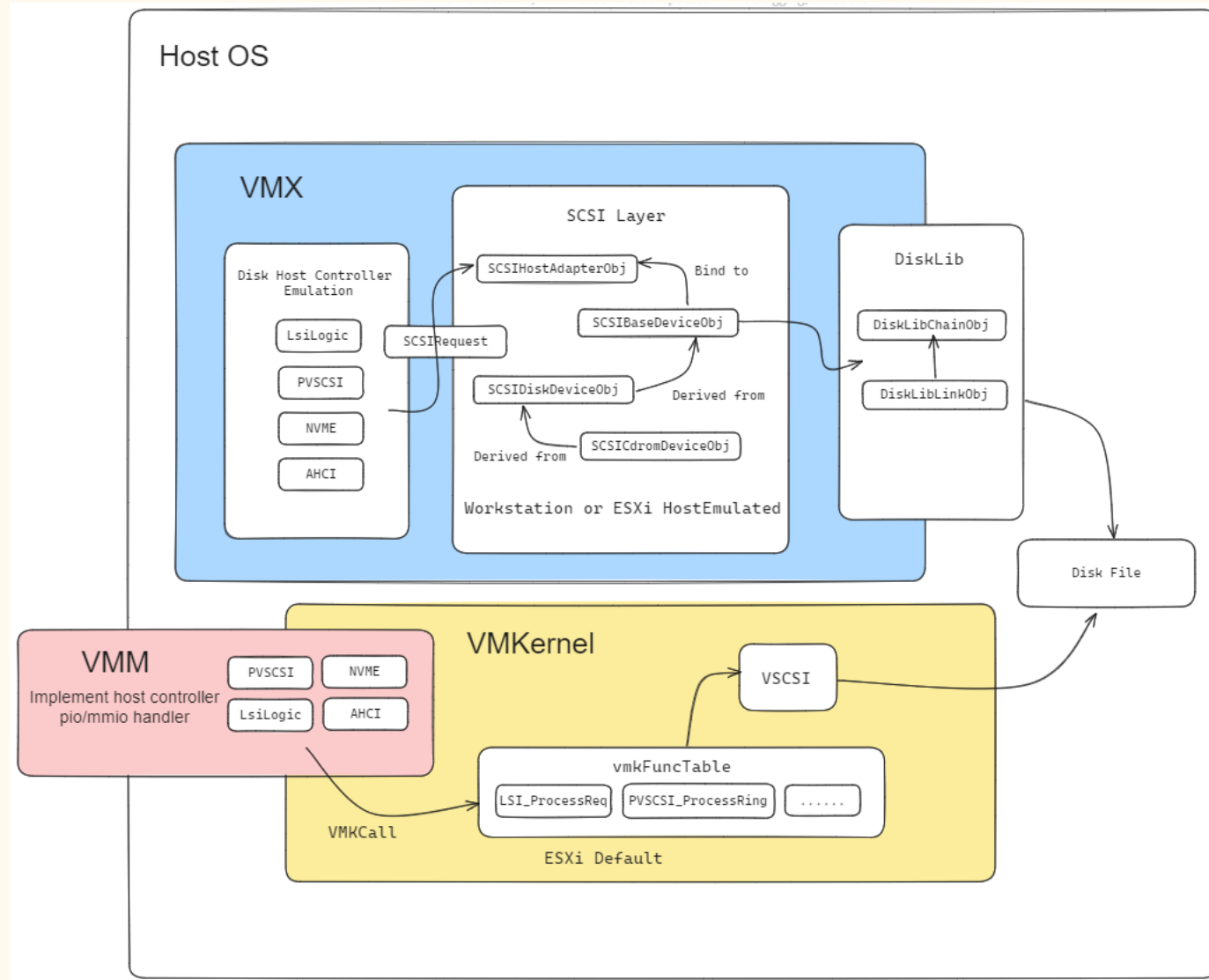


Differences Between ESXi And Workstation

- **ESXi is Type 1 Hypervisor, Workstation is Type 2 Hypervisor**
- **Compared with ESXi, Workstation allows for a more detailed configuration of devices on the GUI page and is compatible with more devices**
- **The data flow direction of device emulation in ESXi is different from that in Workstation**



SCSI Emulation Architecture



SCSI Data Flow Transmission Direction

- **Transmission of SCSI data stream in Workstation**

```
}  
else                                     // MPI_FUNCTION_SCSI_IO_REQUEST  
{  
    LSILogicImplProcessSCSIIOMessage(a1, (__int64)&scsi_io_request, v7, 0, v12, v13);  
}  
return PhysMem_Release(v19);
```

```
if ( *(_BYTE *)(a1 + 0x4F0) )             // hostedEmulation_flag  
    return LSILogicHostedProcessSCSIIOMessage(a1, (__int64)scsi_io_request, a3);  
return result;
```

```
*(&userRpcBlock + 3) = a3;  
*(__int64 *)((char *)&userRpcBlock + 36) = *(_QWORD *)a2;  
*(__int64 *)((char *)&userRpcBlock + 44) = *(_QWORD *)a2 + 8;  
*(__int64 *)((char *)&userRpcBlock + 52) = *(_QWORD *)a2 + 16;  
*(__int64 *)((char *)&userRpcBlock + 60) = *(_QWORD *)a2 + 24;  
*(__int64 *)((char *)&userRpcBlock + 68) = *(_QWORD *)a2 + 32;  
*(__int64 *)((char *)&userRpcBlock + 76) = *(_QWORD *)a2 + 40;  
*(__int64 *)((char *)&userRpcBlock + 84) = *(_QWORD *)a2 + 48;  
*(_DWORD *)&userRpcBlock + 23 = *(_DWORD *)a2 + 56;  
UserRPC(377, 280, v4, v5, v6, v7);
```



SCSI Data Flow Transmission Direction

- **Transmission of SCSI data stream in ESXi**

```
if ( lsilogic_sharedArea->cifvalue <= 0x7F )
{
    if ( lsilogic_sharedArea->hostedEmulation_flag )
        LSILogicHostedProcessSCSIIOMessage(lsilogic_sharedArea, scsi_io_request, guest_phy_mem);
    else
        LSILogicVMKProcessSCSIIOMessage(lsilogic_sharedArea, scsi_io_request, guest_phy_mem);
    return PhysMem_Release((__int64 *)&v14.field_0);
}
```

```
{
    VMK_Call_1Args(0x77u, (unsigned int)lsilogic_shared->adapter_idx);
}
return 0LL;
}
target_adapter_lsilogic_req_num_ring->status = 3;
result = VMK_Call_1Args(0x76u, (unsigned int)lsilogic_shared->adapter_idx);
```



SCSI Data Flow Transmission Direction

- **Different code paths present different attack surfaces**

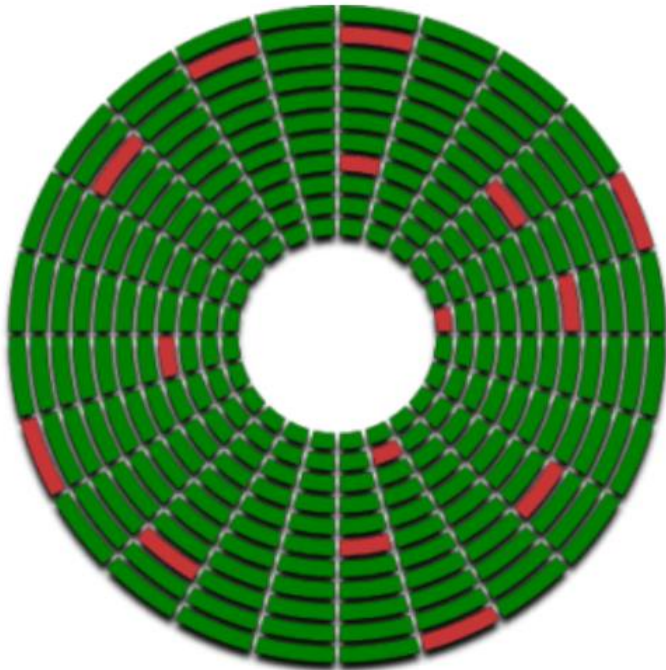


CVE-2024-22273 - Out-of-Bounds Read/Write



CVE-2024-22273 - Code Path to VMX

- **The disk verifier is responsible for detecting whether the disk has bad sectors**
- **VMware implements a disk verifier mechanism**



CVE-2024-22273

- The “Write(16)” command can write data to the specified 64-bit address

Table 219 WRITE (16) command

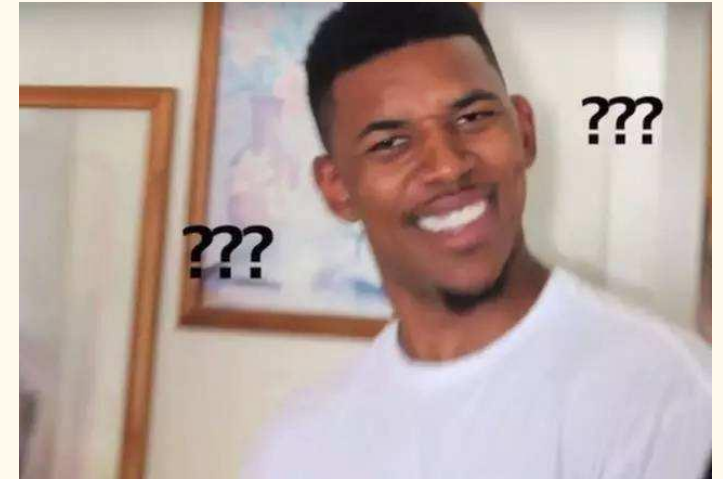
Bit Byte	7	6	5	4	3	2	1	0
0	OPERATION CODE (8Ah)							
1	WRPROTECT			DPO	FUA	Reserved	Obsolete	DLD2
2	LOGICAL BLOCK ADDRESS							
...								
9								
10	TRANSFER LENGTH							
...								
13								
14	DLD1	DLD0	GROUP NUMBER					
15	CONTROL							



CVE-2024-22273

- Normally, the access range of a “Write” or “Read” command is limited according to the disk capacity

```
if ( !*( _QWORD * )( v17 + 0xD8 ) )
{
    v19 = 4 * v5; // 4 * Disk capacity
    v20 = (void *)UtilSafeMalloc1(v19);
    *( _QWORD * )( v17 + 216 ) = v20; ←
    memset(v20, 255, v19);
}
v21 = **(_BYTE **)(a1 + 0x28);
if ( ((v21 - 0xA) & 0x5F) == 0 || ((v21 - 8) & 0x5F) == 0 )
    *( _DWORD * )( *( _QWORD * )( v17 + 216 ) + 4 * v15 ) = v18;
```



CVE-2024-22273

- The “Write(16)” command can be used to write any data to any address

```
do
{
    v15 = v8 + *(_QWORD *)(a1 + 0x70);
    v16 = sub_140604080(v13, v6);
    v17 = *(_QWORD *)(a1 + 16);
    v18 = v16;
    if ( !*(_QWORD *)(v17 + 0xD8) )
    {
        v19 = 4 * v5; // 4 * Disk capacity
        v20 = (void *)UtilSafeMalloc1(v19);
        *(_QWORD *)(v17 + 216) = v20;
        memset(v20, 255, v19);
    }
    v21 = **(_BYTE **)(a1 + 0x28);
    if ( ((v21 - 0xA) & 0x5F) == 0 || ((v21 - 8) & 0x5F) == 0 )
        *(_DWORD *)(*(_QWORD *)(v17 + 216) + 4 * v15) = v18; // Heap Overflow
    v6 = v22;
    ++v8;
    v5 = v23;
    v13 += v14;
}
while ( v8 < *(_QWORD *)(a1 + 120) );
```



CVE-2024-37086 - Out-of-Bounds Read



CVE-2024-37086 - Code Path to VMKernel

- The “UNMAP” command allows one or more Logical Block Addresses to be unmapped



CVE-2024-37086

Table 204 UNMAP command								
Bit Byte	7	6	5	4	3	2	1	0
0	OPERATION CODE (42h)							
1	Reserved							ANCHOR
2	Reserved							
...								
5								
6	Reserved			GROUP NUMBER				
7	(MSB) _____ PARAMETER LIST LENGTH							
8	(LSB)							
9	CONTROL							



CVE-2024-37086

Table 205 UNMAP parameter list

Bit Byte	7	6	5	4	3	2	1	0
0	(MSB)	UNMAP DATA LENGTH (n-1)						(LSB)
1								
2	(MSB)	UNMAP BLOCK DESCRIPTOR DATA LENGTH (n-7)						(LSB)
3								
4		Reserved						
...								
7								
UNMAP block descriptors								
8		UNMAP block descriptor [first] (see table 206)						
...								
23								
...								
n-15		UNMAP block descriptor [last] (see table 206)						
...								
n								



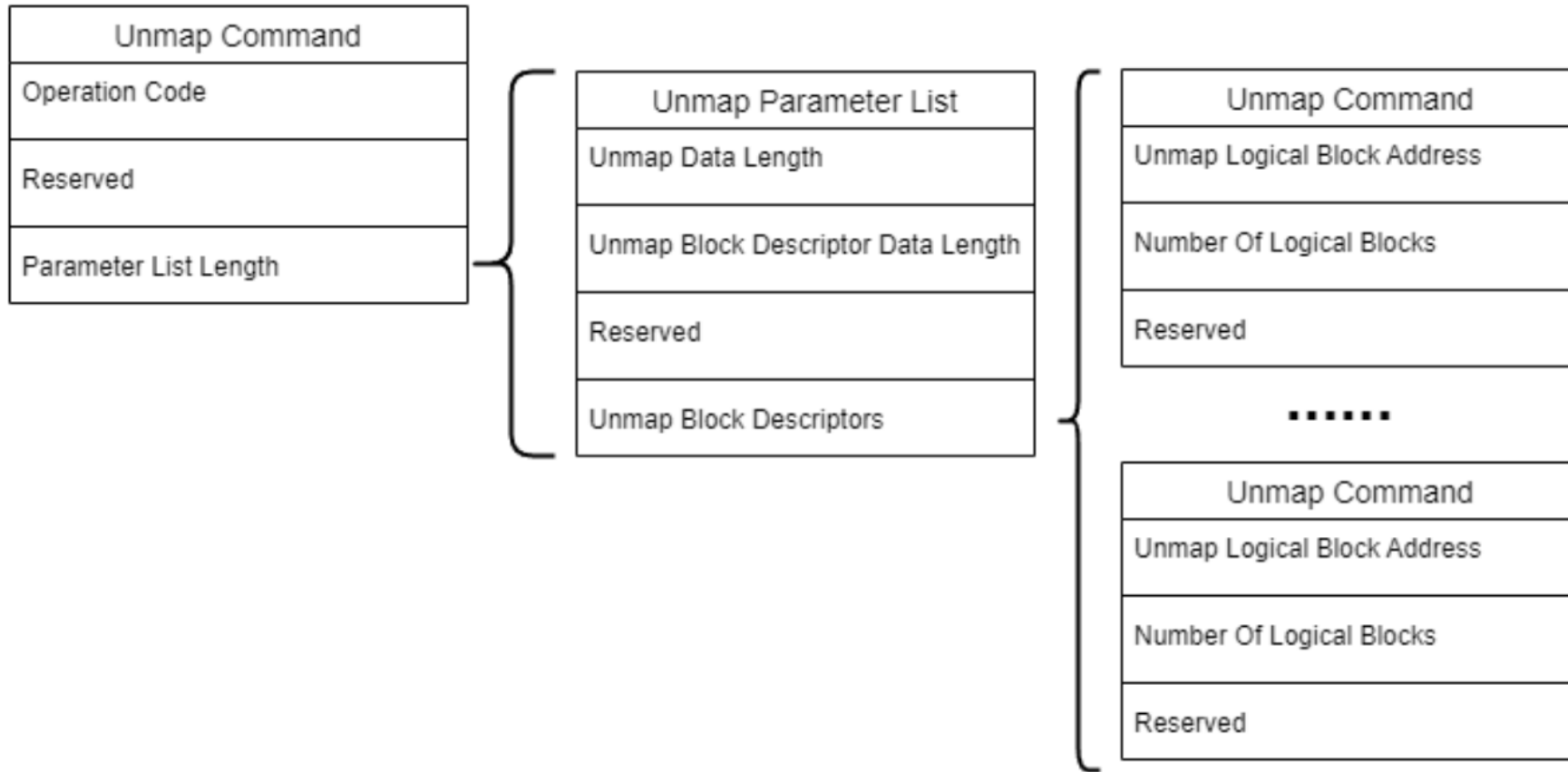
CVE-2024-37086

Table 206 UNMAP block descriptor

Bit Byte	7	6	5	4	3	2	1	0
0	(MSB)							
.....	UNMAP LOGICAL BLOCK ADDRESS							
7	(LSB)							
8	(MSB)							
....	NUMBER OF LOGICAL BLOCKS							
11	(LSB)							
12	Reserved							
.....								
15								



CVE-2024-37086



CVE-2024-37086

- **Verify before using the “UNMAP” command**

```
ret_code = VSCSI_CheckUnmapCmd(vscsiHandle, token, SCSIIO_Command);  
if ( ret_code )  
    goto LABEL_48;
```

```
if ( Parameter_List_Length )  
{  
    if ( Parameter_List_Length > 7uLL )  
    {  
        Parameter_List = (Parameter_List *)VSCSI_Alloc(Parameter_List_Length);  
        Parameter_List_1 = Parameter_List;  
        if ( !Parameter_List )  
        {  
            ret_code = 0xBAD0014;  
            v14 = 8;  
            goto LABEL_6;  
        }  
        if ( !(unsigned __int8)Util_CopySGData(  
            (Inquiry36Response *)Parameter_List,  
            &SCSI_Command->sg_struct,  
            1,  
            0,  
            0,  
            Parameter_List_Length) )  
        {  
            ret_code = 0xBAD0014;  
            v14 = 8;  
            goto LABEL_6;  
        }  
    }  
}
```



CVE-2024-37086

- Forgetting to check the correlation between “Parameter List Length” and “Unmap Block Descriptor Data Length”

```
if ( (v22 || unmap_block_desc_num <= vscsiHandle->UnmapConfig.max_block_desc)
    && (unmap_block_descriptor_data_length & 0xF) == 0
    && (unsigned __int16)__ROL2__(Parameter_List_1->unmap_data_length, 8) == unmap_block_descriptor_data_length
    + 6LL )
{
    if ( !(unmap_block_descriptor_data_length >> 4) )
    {
        ret_code = 0;
        VSCSI_Free(&Parameter_List_1);
        return ret_code;
    }
    unmap_descriptor = &Parameter_List_1->unmap_descriptor;
    while ( 1 )
    {
        unmap_logical_block_address = _byteswap_uint64(unmap_descriptor->unmap_logical_block_address);
        number_of_logical_block = _byteswap_ulong(unmap_descriptor->number_of_logical_block);
        if ( !v22 && vscsiHandle->UnmapConfig.field_0 < number_of_logical_block )
            break;
        if ( unmap_logical_block_address + number_of_logical_block > vscsiHandle->numBlocks )
        {
            v24 = v8;
            v25 = 33;
            goto LABEL_24;
        }
        if ( &Parameter_List_1->unmap_descriptor + unmap_block_desc_num == ++unmap_descriptor )
            goto LABEL_34;
    }
}
```



CVE-2024-37086

- Use “Parameter List Length” as the length

```
do
{
    if ( a1a.Parameter_List > (Parameter_List *)p_unmap_block_descriptor
        || (char *)a1a.Parameter_List + a1a.parameter_list_length <= (char *)p_unmap_block_descriptor )
    {
        VSCSI_Free(&a1a);
        ret_code = Async_EndSplitIO((token_ **)a2, 0, 0, v3);
        if ( !ret_code )
            return ret_code;
        v157 = 0;
        v139 = 0LL;
        if ( ret_code == 0xBAD0002 )
            goto LABEL_448;
        goto LABEL_352;
    }
    number_of_logical_block = p_unmap_block_descriptor->number_of_logical_block;
    unmap_logical_block_address = p_unmap_block_descriptor->unmap_logical_block_address;
    a1a.p_unmap_block_descriptor = ++p_unmap_block_descriptor;
    v144 = _byteswap_ulong(number_of_logical_block);
    v145 = _byteswap_uint64(unmap_logical_block_address);
}
while ( !v144 );
v138 = Async_PrepareOneIO((__int64 *)&a2->field_0, a4);
```



CVE-2024-37086

- **Size variables are affected by “Logical Block Size”**
- **Unchecked “Logical Block Size” will cause Out-of-bound write**

```
if ( size )
{
    v15 = &v63;
    curpos = Page_Start / v14;
    end = size + curpos;
    do
    {
        *(_DWORD *)v15 = curpos++;
        v15 += 3;
    }
    while ( curpos != end );
}
```



New Attack Surface Impact

- **Modify the existing sandbox protection mechanism**
- **Elevate the current process privileges**
- **Virtual Machine Escape**





Thank You!

Twitter: TianGongLab

